一.网络

1.TCP

1.TCP 三次握手

第一次握手：客户端发送同步标志 syn（syn=j）到务器，进入 SYN_send 状态，等待服务器确认；

第二次握手：服务端收到 syn 包，确定用户的 SYN，同时自己发送一个 ACK+SYN（syn=k,ack=j+1），服务器进入 SYN_recv 状态；

第三次握手：客户端接收到服务器发的 ACK+SYN，向服务端发送确认包 ACK（ack=k+1）（确认标志），发送完毕，都进入 established 状态，完成三次握手。

2.TCP 四次挥手

第一次挥手：A---B FIN。客户端 A 告诉服务端 B 自己数据传输完了，请求关闭；

第二次挥手：B---A ACK。服务端 B 告诉 A，收到你要请求关闭的信息；

两种情况，一种 B 的数据没发完，一种 B 的数据发完了（所以才是四次）

第三次挥手：B---A FIN；

第四次挥手：A---B ACK。

3.为什么需要三次握手，为什么不能两次握手

既要保存数据可靠传输,又要提高传输的效率；

如果两次握手就能建立连接的话，考虑这样的情况：第一次发送的时候有网络延迟，根据超时重传，客户端会再发一次，假设第二次成功了，那么将建立连接并进行数据传输，数据传输玩之后关闭连接，这个时候第一次发送的请求到达服务端，服务端会以为客户端还有数据要发送，会一直等待客户端传递数据。（如果有大量的这种情况，会将服务器的半开闭连接队列占满，这也是 dos 攻击的原理）

4.DOS 的攻击原理以及如何预防

DOS 攻击（拒绝服务），DDOS 攻击（分布式拒绝服务），DDOS 攻击是利用多个肉鸡进行 DOS 攻击。

最常见的攻击方式，SYN 攻击，向服务端发送大量的 SYN，完成前两次握手，将半开连接队列占满，这样会阻止其他合法用户的访问。

攻击预防：（1）扩充带宽

（2）隐藏服务器真实 IP，上云等

（3）确保服务器软件没有漏洞，删除未使用的服务，关闭未关闭的端口。

5.TCP 和 UDP 的区别以及适用场景

（1）TCP 依靠三次握手建立连接，是可靠的传输，而 UDP 是无连接的传输协议，即传输数据的之前并不需要建立握手；

（2）TCP 需要消耗的资源多，UDP 需要消耗的资源少。TCP 首部 20 字节，UDP 只有 8 个字节；

（3）TCP 保证数据准确性以及数据的顺序，UDP 不保证；

（4）TCP 只能点到点，而 UDP 支持一对一，一对多，多对多。

TCP 优点：准确可靠消耗多（传输文件）

UDP 优点：无连接速度快（直播）

6.TCP/IP 五层模型

速记：无数王传应

物理层：透明传输比特流，双绞线等

数据链路层（ppp）：接受 ip 数据报并通过网络发送。封装成帧

网络层（arp、ip）：负责相邻计算机之间的通信，点到点的传输（主机或路由器），ip 寻址

传输层（tcp、udp）：建立 tcp 连接（三次握手）

应用层（dns、http）：提供常用的应用程序，比如电子邮件、文件传输访问、远程登陆，dns 解析成 ip 并发送 http 请求

七层：

会话层：建立、管理、终止会话

表示层：数据的表示、安全、压缩

应用层：网络服务与最终用户的一个接口（HTTP，FTP，DNS）

7.HTTP 协议和 TCP 协议有什么不同

两种协议是不同层面上的，HTTP 是应用层上的，TCP 是传输层上的。TCP 单纯的建立连接，不涉及我们需要请求的实际数据，是简单的传输。http 是用来收发数据的，即实际应用上的。

TCP 是底层通讯协议，定义的是数据传输和连接方式的规范。
HTTP 是应用层协议，定义的是传输数据的内容的规范。

8.TCP 可靠的保证

确认和重传机制：在发送一个数据之后，就开启一个定时器，若是在这个时间内没有收到发送数据的 ACK 确认报文，则对该报文进行重传。这个定时器的时间的设置要动态的，不能太小也不能太大。（还有丢包、校验失败等情况）

流量控制：窗口和计时器，TCP 窗口中会指明双方能够接收和发送的最大数据量

数据排序：专门的序列号 SN 字段

拥塞控制：

TCP 慢启动：传输数据或者丢失重发时，对网络实际容量进行试探，避免发送过量数据造成拥塞。

拥塞避免

快速重传

快速恢复

9.TCP 报文

TCP 报文由 TCP 首部和 TCP 数据组成

TCP 首部：20 字节的固定长度和可变长字段

源端口，16 位，数据发起者的端口号

目的端口，16 位，数据接收方的端口号

封包序号，32 位，为每一个封包分配一个序号，保证网络传输的顺序性

确认号，32 位，用来确认确实有收到相关封包，内容表示为期望收到下一个报文的序列号，用来解决丢包问题

数据偏移

保留字段

状态控制码

10.dns

在应用层，用来查询 ip 地址，将 url 转化为 ip。

查找顺序，浏览器缓存->本机缓存->host

2.HTTP

1.http 特点（无连接无状态）

（1）简单快速，资源的 url 是固定的

（2）灵活，在 http 协议的头部有一个数据类型，通过一个 http 协议能完成不同种类的传输。

（3）无连接，连接过一次之后就释放

（4）无状态，服务端无法确认连接者是否访问过

2.HTTP 报文

请求报文：请求行（http 方法，http 地址，协议），请求头（host、origin、描述请求体），空行（分割请请求头和请求体），请求体（数据部分）

响应报文：状态行（http 版本号，状态码），响应头（服务器基本信息、cookie），空行，响应体（请求得到的具体数据，一般为 html 文件内容）

3.http 方法

get：获取资源

post：改

put：增

delete：删

head：获得报文首部

4.get 和 post 的区别

从应用上来说，get 用于从服务端获取数据（静态资源等等），post 用于向服务器提交数据（增删改查）。

（1）长度限制。get 是通过 url 来提交数据的，因此可提交的数据量和 url 所能达到的最大的长度有关（事实上 http 协议对于 url 的长度没有限制，限制 url 长度的是浏览器和服务器的配置参数）。post 受服务端参数配置和内存大小的限制。

（2）get 通过 url 来发送，是明文的，会在 url 后面显示参数；post 通过请求体发送请求，也是可以通过

抓包工具获取到，但是也是明文的。post 的参数是不会被保存在浏览器历史和服务器日志中。get 请求会被浏览器自动缓存，而 post 不会，除非主动设置。所以，如果使用 get 来发送登陆的信息，那么别人可能通过浏览器历史纪录就能查到。
（3）get 在回退是无害的，post 会再次提交请求，get 幂等。
综上，登陆或者删除等应该用 post，get 请求一般只用来请求数据

5.持久连接

http 采用的请求-应答模式，非 keep-alive，每个请求和应答中用户和服务器都要新建一个连接，完成后断开。

http 是支持持久连接的（http1.1 版本才支持）。当使用 keep-alive 模式时，使客户端到服务端的连接持久有效，当出现后续请求时，keep-alive 功能避免了建立或者重新建立连接。

6.管线化

在持久连接下，将请求一次性发送过去，然后一次性返回。

特点：（1）管线化机制通过持久连接完成的，http1.1+；

（2）get 和 head 可以进行管线化，post 有所限制；

（3）初次建立连接时，不应该启用管线机制，因为不确定服务端是否支持管线化

注意：开启管线化可能不一定会带来大幅度的性能提升，所以现代浏览器默认时关闭。

7.常见的请求头

（1）cookie

（2）cache-control 缓存

（3）date 请求发送的日期和时间

（4）range 只请求实体的一部分，指定范围（206）

8.说一下 https

就是加强版的 http，因为 http 的传输都是未加密的，这样当其他人获取到服务器和客户端之间的报文，就可以直接读懂其中的信息。客户端验证服务端的合法性。

第一步：TCP 三次握手

第二步：客户端发起请求，服务器生成公匙 S 和私匙 S，服务器将证书信息返回给客户端，包含服务端自己生成的公匙 S，客户端通过浏览器内置根证书来验证证书的合法性。如验证成功，则返回一串随机数（密匙 C），将密匙 C 通过公匙 S 进行加密，生成密匙 C*。客户端将私匙 C*发送到服务端，服务端通过密匙 S 进行解密成私匙 C，再将数据用私匙 C 进行加密，在将数据发送到客户端。客户端接收到通过密匙 C 加密的信息，在本地用密匙 C 来解密。

缺点：传输所需的资源多，时间长；ssl 证书需要费用；端口不同，一个 80，一个 443

9.http1.1 和 http2 之间的区别

（1）http2 是多路复用，只需要一个连接就可以实现并行。在 http2 中，使用了单一长连接，不管要传输的数据有多少，只会建立一次 TCP 连接。在逻辑上，http 的请求可以分成不同的通道，又通过二进制分帧，这样就可以保证多路复用。

（2）二进制分帧，多路复用中采用的技术，将请求消息分解为更小的帧，比如将 head 封装在 head 帧里面，body 封装在 body 帧里面，可以将多个请求都分解，然后乱序发送到服务端，服务端再进行拼接。

（2）http2.0 使用了报头压缩，降低了开销，采用了 Hpack 算法。

（3）server push，缓存推送，当用户请求一个资源 x 的时候，服务器直到它很可能再请求 y 资源，这时，服务器主动得将 y 资源推送到本地的缓存中，以备将来使用。

10.状态码

1. 提示信息
2. 成功
3. 重定向
4. 客户端错误
5. 服务器错误

200.客户端请求成功

206.带了限制（range 比如说，0-1000 的字节）的 get 求情，服务器完成了（只有在响应头中）（当请求的视频或者音频很大时，会返回 206）

301.页面永久转移至新的 url，比如更换了域名，爬虫友好

302.页面临时转移至新的 url，比如未登陆临时跳转到到登陆页面，搜索引擎认为 302 是作弊的手段，会降低网站权重

（能使用 301 的情况下不要用 302）

304.服务端告诉客户端，原来缓存的文档还能使用。（从协商缓存中返回的，不是强缓存，强缓存不和服务端通信）

400.客户端语法错误，不能被服务器理解
401.请求未授权，请求用户验证。
403.请求被禁止，用户验证未通过（只能通过服务器访问那个地址）。
404.资源不存在
500.服务端错误。
503.服务器临时过载或者宕机。
504.网关超时,后端电脑之间 IP 通讯缓慢造成的,如果 web 服务器由某一个网站托管,只有负责哪个网站的设置的管理员才能解决这个问题.可能是在处理 HTTP 请求的一连串电脑中,有一个节点坏了。

3.缓存与存储

1.说一下 http 缓存

缓存：在浏览器本地的副本，比如说一个图片缓存到本地。
（1）强缓存：在多长时间之前直接用缓存，不请求服务器。cache-control 和 expires(等同于 cache-control 中的 max-age)
（2）协商缓存：有缓存时也要协商一下到底从不从缓存拿数据。
Last-modified:服务器下发资源文件时的时间；
Etag: 是一个代表文件的 hash 值，表示在服务器的那个文件。
优先级：cache-control>expires>etag>last-modified

2.缓存具体属性

（1）cache-control，取值可以为下面的 cache-control：
private：cache-control 的默认取值，表示仅客户端可以缓存，服务端不可缓存。
max-age=5，过期时间，单位秒，表示访问服务器上的内容之后的 5 秒内都不会再去服务器访问了，而是直接访问缓存。
public：客户端和服务端都可以缓存。
no-cache: 必须先访问服务器
no-store：绝对禁止缓存
（2）expires，
过期时间。这个属性用来告诉缓存器，这个副本在多长时间内时新鲜的.时间必须时格林威治时间,而不能是客户端时间.
（3）last-modified
值为一个时间，代表了上一次从服务器获得数据的时间。如果在这个时间之后服务器的资源没有修改，则直接调用缓存。
缺点：1.只能精确到秒级 2.多个服务器的时间可能不一致 3.旧文件覆盖新文件，不能识别
（4）ETAG，给文件或者资源一个 hash 值，这个值表示了这个文件的独特标志，第一次访问某个 url 时，服务器会给发到本地的文件一个 etag，第二次访问某个 url 时，请求头上会带有 etag 属性。服务器接收到这个 etag，检查 etag 关联的文件或者页面自上次客户端请求后是否修改过。作为 last-modified 的补充。

3.缓存配合使用

缓存一般是配合着使用的
last-modified+expires:last-modified 用来解决强制刷新 expires 不起作用，会发 http 请求。
expires 用来解决 lat-modified 就算有缓存也会发请求。
强+协商

4.espires 缓存设置

html 页面使用：<meta http-equiv="Expires" content="GTM">
对于图片、css 等文件要在服务端设置 response 的响应头

5.cookie 与 session 的区别与联系

cookie：本地存储数据
session：服务器端存储用户数据的
他们的联系：每个用户都会再服务器上生成不同 id 的 session，当用户下次访问时，会将 cookie 随 http 一起发给服务端，服务端找到 id，然后找到 session，这样就可以还原上一次的访问状态。

6.cookie 中的字段

name：cookie 的名称
expires/max-age:过期时间，如果不设置的话，那么 cookie 的存活时间是向 session 看齐的。

value:cookie 的值
domain:可以访问此 cookie 的域名
secure:设置是否只能通过 https 来传递这条 cookie
size：cookie 的大小

7.session 具体是怎么存储数据的

（1）客户端向服务器发送一次请求，想让服务端将自己的名字设置到会话中。

（2）服务器调用 request.getSession 方法，产生 session 对象，用于跟踪用户状态，同时，给 session 对象分配一个唯一的标识。并以 sessionId 为键，session 对象为值，可以存在内存中，也可以持久化到数据库中

（3）产生响应，将 sessionid 放入 cookie 中发送到客户端。

（4）当客户端再次请求服务器，会将携带 sessionid 的 cookie 发送给服务器。服务器解析 cookie，得到 sessionId，从 map 中查找到 session 对象，进而跟踪状态。

8.没有 cookie 如何使用 session

由于 session 存在 cookie 中的是 sessonId，如果没有 cookie，那么可以将 id 存放在其他地方，比如 url 后面。或者可以和 token 一样的方式

9.token 是什么

正常的 cookie+session 验证身份，每次请求都要在服务端中根据 cookie 的 sessionid 来获取 session，然后再查找用户状态

token 是服务端下发给客户端的，服务端接收到请求，通过 location 获取客户端的设备信息，设置一个 token 保存这个设备信息（一般是客户端的 mac 地址），然后将这个值保存在 session 中。再下发给客户端，可以保存在 locastorage 或者 cookie 中。

10.web 本地存储的方式

Sessionstorage：用于本地存储一个会话中的数据，只有在同一个会话的页面才能够使用，同一个浏览器标签里

Localstorage：持久化的本地存储，除非主动删除数据否则不会过期；

两者称为 webstorage：本地的存储，5M 以上。

Cookie：存在本地，存储数据发送到服务端，cookie 始终在同源的 http 请求中携带。不超过 4k。设置过期时间，不管浏览器有没有关闭。

11.Webstorage 对比与 cookie 的优点：

1.存储空间大。
2.存储内容不发送到服务器端。q
3.更多的接口，cookie 需要自己封装。
4.独立的存储空间，每个域有独立的存储空间，不会造成数据混乱。

12.cache-control 流程

第一次返回的时候，会将

13.ETAG 和 last-modefied 流程

4.通信与跨域

1.Ajax 的原理

第一步：实例化一个 XHR 对象
第二步：回调函数，xhr.onreadystatechange()
第三步：open 函数，第一个参数确定方式、第二个参数 url、第三个参数是否异步，默认 true
第四步：send 函数，发送（get 则没东西，post 为要发送的数据）
第五步：close 函数，关闭

2.readystate:

0: 请求未初始化。xhr 对象已经创建，但未初始化（使用 open 里面的东西进行对象状态的设置）

1: 服务器连接已建立，正在向服务器发送请求。调用了 open()完成 xhr 对象的初始化，并且调用了 send 开始发送请求

2: 请求已接收。调用 send()，请求已经发送完成，已经接收完全部响应式数据

3: 请求处理中。正在解析数据，在根据 respons 头部的 mime 类型把数据转换成 responsbody 或者 responsText 等内容

4: 请求已完成，且响应已就绪。表示解析已经完成，可以通过属性调用了

3.什么是跨域

因为 js 处于安全方面的考虑，有一个同源策略限制，也就是限制从一个源到不同源的访问和操作。同源：协议+域名+端口。

4.跨域的方法以及优缺点

1.JSONP，由于调用 js 文件时，不受跨域的影响）。实际上就是把服务端生成的 js 嵌入本地的 html 文件。

缺点：只能使用 get 请求，不能注册 success 等事件监听函数，不能很容易的确定请求是否成功或者失败，不安全。

优点：支持老式浏览器，以及可以向不支持 cors 的网站请求数据。

2.CORS，跨域资源共享，实际上就是变种的 ajax，在请求头部有 origin 字段，包含了客户端的地址。请求的时候浏览器先发一个“预检”请求，包含了客户端地址，服务端判断是否是允许访问的，如果允许，那么浏览器才发正式的 http 请求。

缺点：双端必须都允许 cors，特别是服务端。

3.postMessage：H5 新增（跳转链接的时候传值）（woker 传值 ）

从 A--->B

A 中：（B 的）window.postmessage(“data”,url);

B 中:新建一个 message 监听函数，取到 event.origin（=A 的 url），event.source（=A 窗口），event.data（要发送的数据）

窗口间通信。

4.Websocket: h5 新增(web 实时应用)协议

先声明一个

send()发送数据-->.data-->close()关闭连接

websocket 协议解决了 http 通信只能由客户端发起的问题。

优点：建立在 TCP 协议之上，服务端实现比较容易；没有同源限制；数据格式比较轻量;http 协议做服务器推送很麻烦，用 websocket 可以用来做轮询或者服务器推送，因为它支持服务端发起通信

（5）hash（跳转链接的时候传值）

url 中#后面的东西，hash 改变页面是不会刷新的，所以可以用来做跨域通信。

```
//在当前页面 A 中通过 iframe 嵌入一个跨域的页面 B。
//在 A 中
var B = document.getelementbyid('ifram');
B.src = B.src+'#'+'data';
//在 B 中
window.onhashchange = function(){
        var data = window.location.hash;
}
```

（6）图片 ping（计数）

使用 img 标签，请求数据通过字符串的形式发，而响应可以是任何内容。（只能发送 get 请求，浏览器无法获取响应数据，只适合用于浏览器和服务器之间的单向通信）因此，一般用来做统计点击次数。

5.Jsonp 的具体过程

（1）客户端注册一个 callback，然后把 callback 的名字传给服务器（callback）

（2）服务器生成 json 数据，然后以 js 语法的方式生成一个 function

（3）将 json 数据直接以入参的方式，放置到 function 中，生成一段 js 语法的文档，返回给客户端。

（4）客户端浏览器解析 script 标签，并执行返回的 js 文档，此时数据作为参数，传入到客户端预先定义好的 callback 函数里（动态执行函数）

callback 就是回掉函数的名字。

local html

```
<script>
var handle = function(data){
    alert(data.result)
}
//这里后面都是为了在远程动态获取本地的回调函数的名字
// 提供jsonp 服务的url 地址（不管是什么类型的地址，最终生成的返回值都是一段javascript 代码）
var url =
```

```
"http://flightQuery.com/jsonp/flightResult.aspx?code=CA1998&callback=handler";
// 创建script标签，设置其属性
var script = document.createElement('script');
script.setAttribute('src', url);
// 把script标签加入head，此时调用开始
document.getElementsByTagName('head')[0].appendChild(script);
//script.onerror()



</script>
//生成的是这样的：
<script src="http://../remote.js"></script>
```

remote.js

```
handle({
"result": "数据"
})
```

6.CORS 的具体过程

（1）如果是 get 等简单请求，那么就在头信息中，增加一个 origin 字段。origin 表示来源（域名+端口+协议），服务器根据这个值来判断是否同意请求。如果不允许，那么会返回一个错误，这个错误会被 xhr 的 onerror 回掉函数捕获。

（2）如果是 put，head（类似 get 请求但是不返回消息体，适用于速度和带宽好的），则要先发送一个预检请求。用 setRequestHeader 自定义头信息。

7.postmessage 有哪些好处

异域的两个客户端之间共享数据，其他方式都是客户端和服务端。（hash 也是）

8.json 和 xml 的优缺点比较

json 优点：轻量级，易于人阅读和编写，也易于机器解析和生成。

json.stringfy（）将 json 对象转化为 json 类型字符串。

json.prase（）将 json 类型的字符串转化为 json 对象。

在 ajax 使用 json 的基本方法是通过前台载入后台声明 js 对象的字符串，用 eval 方法将它转化为实际的对象，最后通过 Dhtml 来更新页面信息

它是 js 的原生格式，因此在 js 中处理 json 数据不需要任何特殊的 api 或工具包，而且效率非常高.

xml 优点:格式统一,符合标准,容易与其他系统进行远程交互,数据共享比较方便.

json 缺点:没有 xml 那么具有通用性

xml 缺点:xml 文件庞大,文件格式复杂,传输占带宽.

客户端和服务端都需要花费大量的资源来解析 xml 文件.

不同浏览器解析 xml 的方式不一样,需要重复编写很多代码.

9.json 的方法

JSON 中，有两种结构，对象和数组。数组格式就是多个对象。在数据传输过程中，JSON 是以字符串的形式传递的，而 JS 操作的是 JSON 对象。

JSON 字符串转化为 JSON 对象：：： JSON.parse(要转的)JSON.eval（‘(’+要转的+’)’）

JSON 对象转化为 JSON 字符串：：：：JSON.stringfy(json 对象)

10.JSON.parse 和 eval 的区别

都是用来将字符串转化为 JSON 对象或数组的。

（1）eval 的速度更快

（2）eval 会执行字符串中的 js 方法，而 parse 则会报错，所以 parse 更安全。

5.实际使用综合问题

1.在浏览器输入一个 url 发生了什么

1.域名解析：chrome 通过 DNS 查找域名对应的 ip 地址

先查浏览器缓存->系统缓存->路由器（DNS 缓存）->ISP(DNS 缓存)->在 isp 的 DNS 服务器从根服务器域名服务器开始递归搜索最后得到 IP 地址

2.TCP 三次握手；

3.开始通讯，传输数据；

4.服务器返回 html 响应；

5.渲染页面。

2.CDN 原理

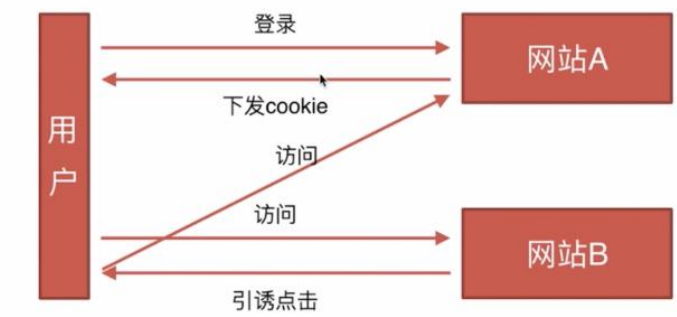
内容分发网路，在现有的 internet 中增加一层新的架构，将网络的内容发布到最接近用户的网络，使用户可以就近获得所需的内容。当用户访问网站内容的时候，通过调度系统将用户的请求发送到离用户最近的缓存服务器上，调度系统使通过负载均衡的方式，综合考虑与用户之间的距离、阻塞情况、流量等因素。

举例来说，当用户点击网站页面上的 URL 时，经过本地 DNS 系统解析，找不到的话就会将解析权交给 CDN 专用的 dns 服务器，CDN 的 DNS 服务器将全局负载均衡设备 IP 地址发给用户，然后用户回像负载均衡设备发出 URL 访问请求，CDN 全局负载均衡设备会根据用户 IP 地址，以及所请求的 URL，选择用户所属区域的区域负载均衡设备，告诉用户向这台均衡负载设备发起请求。

6.安全

1.CSRF

跨站请求伪造。利用接口本身的漏洞，来实现。比如新浪微博的关注 api。



防御措施：

token 验证：放入不能伪造的信息，在 http 请求头中的一个自定义属性中加入一个随机产生的 token，并在服务器端建立一个拦截器来验证这个 token，如果请求中没有 token 或者 token 内容不正确，那么就认为时 CSRF 攻击而拒绝。token 是在用户登陆后产生并放在 session 中的。

Referer 验证：在 http 头中有一个字段叫 referer，记录了该 http 请求的来源地址。

2.XSS

跨站脚本攻击原理：主要原因时过于相信客户端提交的数据，根源在于没有完全过滤掉客户端输入的信息。向页面注入脚本，比如在提交区中或者在 url 上，提交携带 js 的，然后运行 js。

防御：（1）进入输入过滤，将<script>,<iframe>等标签过滤掉

（2）过滤 script 中的事件标签，例如 onclick 等

（3）将 cookie 设置为 http only，避免在 dom 中获取 cookie。

3.SQL 注入

原理：就是通过 sql 命令插入到 web 表单提交或者输入域名，最终达到欺骗服务器执行恶意代码。

防御方法：（1）对输入进行验证。在客户端和服务端都要进行验证，将用户的输入进行检查，判断其是否合理（例如特殊字符、长度等）。

（2）不要动态拼接 sql，使用参数化的 sql；

（2）对输入进行处理。对用户的输入进行加密，然后再与数据库中的内容进行比较。消毒。MD5 进行加密。preparedStatement。

（3）错误信息处理。hacker 可以通过错误信息来猜解表名等信息。

二.DOM

7.DOM 事件

1.DOM 事件的基本概念与级别

DOM0：element.onclik = function(e){}，简单稳定，但是只会在事件冒泡中运行，一个元素一次只能绑定一个事件处理函数，新绑定的会覆盖旧的，e 不能再 ie 中。

DOM2:element.addEventListener(‘click’,function(){},false)，加入了捕获和冒泡

DOM3:element.addEventListener(‘keyup’,function(){},false),增加了监听类别，键盘事件，鼠标事件等等。

第三个参数代表是在冒泡还是捕获阶段执行，如果为 true，事件会在捕获中执行，如果为 false（默认），事件会在冒泡中执行。

2.标准事件模型与 ie 事件模型

```
addEventListener('click',function(){
    alert("我是 button");
}, true);
target.attachEvent('onclick',function(){
    alert("我是 button");
});
```

不同：（1）IE 只支持冒泡，标准事件模型支持捕获和冒泡。

（2）IE 事件类型前面要加 on

（3）标准中如果同一函数被注册多次，则会执行多次；IE 中后面的会覆盖前面的。

（4）IE 中的 this 指向是 window，而标准的是指向所属的元素

3.捕获与冒泡

由于在捕获阶段没有阻止捕获的函数，并且为了兼容 ie，所以事件一般绑定在冒泡阶段。

捕获--->目标阶段-->冒泡

捕获，window，document，html，body,.....目标元素，冒泡则相反

4.阻止默认事件

比如 a 默认绑定跳转的事件，form 里面默认绑定了提交事件。要阻止这些事件的发生。

标准：event.preventDefault()；

IE：window.event.returnValue = false;（标准也可以用这种方法）

```
//兼容写法
function someHandle(event) {
        event = event || window.event;
        if(event.preventDefault)
                event.preventDefault();
        else
                event.returnValue = false;
}
```

5.阻止冒泡

如果父元素和子元素都绑定了事件监听函数，那么，当点击子元素的时候，父元素的方法也会触发。但是我们常常不需要触发父元素的。

标准：event.stopPropagation();

IE：window.event.cancelBubble = true;（标准也可以用这种方法）

```
//兼容写法
function someHandle(event) {
	event = event || window.event;
	if(event.stopPropagation)
		event.stopPropagation();
	else
		event.cancelBubble= true;
}
```

6.其他方法

event.stopImmediatePropagation()：比如点击绑定了两个事件，如果要防止另一个相应，则在第一个里面加上这个。（标准中的，IE 中不需要，默认为覆盖）

event.target：当前点击的元素；id：点击元素的 id；nodeName:点击元素的标签；className:点击元素的 class

event.currentTarget：当前触发事件的元素，即绑定事件的元素

7.事件委托/代理的原理与实现

由于全部子级元素都绑定事件的话，会消耗很大的资源，导致浏览器缓慢，这时，将事件绑定在父级元素上，再通过 event.target 判断触发的点是不是子元素，这就是事件委托。而且，新的自己元素加进来也可以绑定事件。例如无序例表中我们需要点击每个 li，则只要点击给 ul 绑定一个事件监听即可，再判断 ul 中点击的元素是否是 li 即可。

8.事件委托的缺点

（1）事件委托是基于冒泡的，对于不冒泡的事件不支持

（2）层级过多，在冒泡过程中，可能会被某层阻止掉

（3）把很多事件用上代理可能会出现事件误判，例如，在 document 中代理了所有 button 的代理事件，其他人在后面的维护和修改时，可能不知道，造成单击 button 触发两次事件（或者覆盖）。

9.事件捕获的使用场景

用来做全局事件监听，比如要求在某种情况下，要求只要全局有点击事件，就触发某种事件，那么可以在 document 上加事件监听。

10.移除事件绑定

用 removeEventListener()可以移除用事件监听添加的，注意，里面的函数必须是用外部函数。

8.DOM 节点操作

1.创建节点(前面都要加 document.)

createDocumentFragment() //创建一个 DOM 片段

createElement('div') //创建一个具体的元素节点

createTextNode('hello world') //创建一个文本节点

2.appendChild 和 insertBefore 两种插入方式的不同之处

再插入之前必须先创建这个节点。

father.appendChild(") //添加，并且位于末尾

brother.insertBefore(") //插入，并且位于兄弟节点前面

3.其他方法

removeChild() //移除

replaceChild() //替换

getElementsByTagName() //通过标签名称

getElementsByName() //通过元素的 Name 属性的值

getElementById() //通过元素 Id，唯一性

4.完整的创建节点的步骤

（1）创建节点: var el = document.creatElement('div');

（2）设置其他的属性: el.className = 'myclass'等等;

（3）插入: html 中的元素.appendChild(el);

5.如果一个 ul 里面插入 1000 个 li 节点，怎么插入？

插入方法：①先 creatElement 创建节点 li，然后 ul.appendchild()插入。

②先拼接 1000 个 li 的字符串，然后 innerhtml。

最好的方法:使用 creatDocumentFragment()（文档碎片），在里面添加动态元素，再进行 appendchild，这种方法中，文档碎片相当于是占位符，插入的时候会将所有的子孙节点都插入，克服了 innerHTML 的缺点。其实就是先在循环中将 1000 个 li 节点 appenChild 进 fragment，然后再将 fragment 插入到 ul 中。

6.何时使用哪种方式插入

一般情况下，使用 innerHTML 更简单更快，特别在 ie 中，因为 innerHTML 是静态的，不需要操作内存。使用 appendChild 则还要考虑顺序之类的问题。

使用 innerHTML 的限制：

（1）大多数浏览器中 innerHTML 插入的脚本不能运行，IE8 之前的版本可以运行，但是也要满足一些条件。

（2）js 与 HTML 代码耦合程度高，不容易维护。

（3）innerHTML 是整个重写该元素内的所有 HTML 代码，覆盖。

如果少量则先 creatElement 再 appendChild，如果是大量的话先 creatDocumentFagment 再 appendChild。尽量不用 innerHTML（ie 可以考虑）

9.移动端事件

1.如何判断是否是移动端

在 navigator 对象中，包含了浏览器的信息。navigator.userAgent 中申明了 HTTP 请求的用户代理头值。

/Android|webOS|iPhone|iPod|BlackBerry/i.test(navigator.userAgent)

location.host 中包含的是当前 url 的主机名称和端口号

2.常用事件

1.click：和 PC 端不同，有 200-300ms 的延迟。为了双击触发 doubleclick 事件

2.touch 触摸事件

3.swipe 类事件

swipe：手指滑动即会触发

swipeUp、swipeDown、swipeLeft、swipeRight

三.运行机制及优化

10.浏览器内核

1.浏览器的多进程

进程包括主进程（负责协调、主控），GPU 进程（最多一个，用于 3D 绘制），浏览器渲染进程（内核，每个 tap 一个）

2.浏览器的多线程

GUI 线程

JS 引擎线程（所以 js 是单线程的）

事件触发线程

定时器线程

网络请求进程

9.渲染机制

1.网页生成的过程

1.解析 html 树，创建 DOM 树。

自上而下，遇到样式和脚本则会阻塞，外联样式不阻塞

2.解析 css。创建 CSSOM（样式结构体）

每个元素都是以盒模型形式存在的

3.将 cssom 与 dom 树合并，生成 render dom 渲染树，包含每个节点的视觉信息。

4.flow:布局渲染树，结合 layout，计算每个盒模型大小和位置。(Reflow)

5.paint:绘制渲染树。(Repaint)

渲染=第四步+第五步

注意：浏览器解析完一部分就显示一部分，同时，还可能通过网络下载其他内容

2.什么是 reflow，哪些情况触发 reflow 回流

dom 结构中的各个元素都有自己的盒子，这些需要浏览器根据各种样式来计算并根据计算结果将元素放到他该出现的位置，这个过程就是 reflow。速度较慢。

（1）增删改 dom 节点时。

（2）移动 dom 元素，或者动画时。

（3）改变文字大小，改变默认字体

3.如何减少 reflow

（1）通过建立一个队列，可以使用 style.csstext，将要进行的操作封装起来，然后用 csstext 一起执行处理。现代的浏览器都做到了这一步，维护一个队列，把所有引起回流和重绘的操作放进这个队列，等队列中有一定的数量，或者经过特定的时间间隔之后，浏览器就会处理队列，将这些操作一次进行处理，这样就避免了引起多次回流和重绘。

（2）将需要多次进行重排的元素脱离文档流，比如绝对定位等，这样就不会影响其他的。

（3）将要进行多次重排的元素进行 display:none，重排完毕再显示。本质上还是脱离文档流。

（4）避免使用 table 布局，因为 table 中的一个元素进行了 reflow，会导致 table 里面其他的元素都触发 reflow，可以设置 table-layout 为 auto 或者 fixed，让 table 一行行的渲染，从而限制 reflow 的范围。

（5）将会导致回流的取值操作先进行并保存，避免放在循环里面。比如获取当前的真实的 style。getComputedStyle(),或者 IE 的 currentStyle。

（6）使用离线 DOM，而不是真实的 DOM，来改变元素样式，一般用碎片文档（documentFragment）。

4.什么是 repaint 重绘，触发条件

就是将内容画到页面上。速度相比 reflow 较快。

所有的 reflow 都会触发 repaint，有的会触发 repaint 不会触发 reflow：改变颜色等。

5.onready 和 onload 的区别

onready 表示文档结构已经加载完成，不包含图片等，内部实现是 DOMContentLoaded 事件，在 IE 中通过

onload 表示包含图片在内的所有元素都已经加载完成

onready 可以有多个，并且每个都能生效，而 onload 则只有最后一个才能生效

6.style、css、js 的阻塞情况

css 不会阻塞下载，js 会阻塞下载（因为对于 js 来说，可能会修改 dom，所以在当前 js 下载和运行完毕之前，后面的都是没有必要的），但是现代浏览器 js 支持并行下载

css 阻塞后续 js 执行（原因：js 执行前，必须保证前面的 css 都已下载和解析完成，因为 js 执行可能依赖最新样式）

css 不会阻塞 dom 解析，但是会阻塞 dom 渲染

内联 js 会阻塞所有内容的渲染

外联 js 只阻塞其后的内容的渲染

7.使用内联 css 还是外联 css

外联 css 下载的时候需要发起 http 请求，第一次的时候比较慢，但是可以缓存。内联样式不能缓存

因此，对于访问多的页面，使用外联 css 比较好，可以缓存，对于访问比较少的页面，使用内联样式。

但是现代的前端开发，都是模块化，所以写在外联 css 中比较好

8.内联 js 与内联 css 的顺序

如果是在 head 里面重要的 js，那么内联 js 需要放在 css 前面，因为浏览器中内联 js 会阻塞其后面内容的下载，所以把 js 放在前面早点执行完，让后面的下载。

10.JS 运行机制

1.什么是任务队列和 event loop

js 是单线程的。

js 中的任务分为同步任务和异步任务。

任务队列是用来存放异步任务的。

整个过程：JS 运行过程中，将同步任务压入运行栈，直接执行，运行栈为空那么就会检查任务队列。JS 遇到异步任务时，会将异步任务用各种模块（例如 settimeout 可以用 timer 模块）来处理，读取到异步任务处理完毕（例如事件监听到）之后会压入运行队列。

只要运行栈为空，就会去读取任务队列，这就是事件循环。

所以，如果异步任务被放在一个循环里面，那么会先执行作为同步任务的循环，等到循环执行完毕，再连续执行 n 个异步任务。

2.什么时候会开启异步任务

（1）settimeout 和 setinterval

（2）dom（监听事件）

（3）promise

3.什么是 promise，promise 的原理以及链式调用

promise 相当于一个容器，里面保存着某个未来才会结束的事件（异步操作），这个容器提供了统一的 api，用统一的方法处理各种异步操作。

promise 不是一种解决具体问题的算法，只是一种更好的代码组织格式。

原理：在实例化一个 promise 的时候，需要传入两个参数 resolve 和 reject。在 promise 里面执行异步操作，如果成功则用 resolve 返回成功结果，若失败则用 reject 返回失败结果。再用.then 接收成功的结果，用.catch 接受失败的结果。

链式调用：ppp.then(fucntion).catch(function)

4.promise 对象的特点

（1）对象的状态不受外界的影响，只和异步操作的结果来决定。

（2）一旦状态改变，就不会再变，任何时候都能得到这个结果。（两种 pending-->fulfiled 或者 pending-->rejected）。这个时候的状态总成为 resolved（已定性）。若改变已经发生了，再对 promise 添加回调函数，依旧可以获得这个结果，而传统的事件，一旦错过了它，再去监听，是没有结果的。

5.promise 的状态和生命周期

pending（进行中），fulfiled（已成功），rejected（已失败），后两种统称为 resolved

状态只由异步操作的结果来决定

6.promise.all 与 promise.race

promise.all([p1,p2,p3])将多个 promise 实例包装成一个新的 promise 实例，成功则返回一个结果数组，失败则返回最先被 reject 失败状态的值。

promise.race([p1,p2,p3])哪个结果跑的快，就返回哪个结果。

7.call、apply、bind 区别

call 和 apply：让函数在某个指定的对象下执行.第一个参数就是指定的对象，也就是这个函数的执行上下文（this）。比如 add.call(this)，就是再当前环境下执行 add 中的方法。

apply 第二个参数是数组，而 call 则可以有多个参数。

bind 函数不会立即执行，而是返回一个改变了上下文的函数副本。所以 bnd 方法返回的依旧是一个函数。xxx.xxxfunction.bind(yyy)(参数)。需要后面调用。

所以，如果想让函数不立即执行，而是需要在后面调用的话，就用 bind

8.asycn 和 await

asycn 是 generator 的语法糖。

asycn 用来返回一个 promise 对象，await 用来等待 promise 对象。

9.generator

类似于数组，不过数组是只返回一次，generator 可以返回多次，所以用来做异步任务的处理。生成器内部的 yeild 相当于一个 return，yeild 后面的都不会执行，在生成器外面使用.next()会执行下一个 yeild。

10.promise 错误捕获

错误会顺着 then 一层层下去，直到遇到第一个 catch。但是 then 中的第二个方法可能会将错误截胡，如果这个 then 中的第一个方法有错误，那么会覆盖之前的错误。

所以 promise 中 catch 指挥跑出来最近的一个错误。

如果想要将每一个错误都打印，那么需要在每层里面都调用。

11.microtasks 和 macrotasks

任务队列分为两种，一种是 microtasks 微任务，一种是 macrotasks 宏任务。微任务的优先级比宏任务的优先级更高，当微任务队列调用完了，才会去宏任务队列中调用一个宏任务。这时候依次执行这个宏任务包含的运行栈的任务->微任务队列重的任务。

microtasks：process.nextTick->promise；ajax
macrotasks：script;setTimeout;setInterval;I/O;UI 渲染;点击事件

11.web 性能优化

1.优化的方法

1.减少 http 请求：css sprites、lazyLoad、缓存、工程化中图片放在项目中
2.精简资源
3.加快网络：cdn、dns 预解析、静态网页不使用 cookie
4.css 与 js 优化
5.ssr

2.减少 http 请求

频繁的建立连接释放链接，会造成资源的浪费，每个 http 请求都会给服务器和浏览器产生性能负担。

（1）css sprites：图片精灵。合并图片，将所有的零星图片整合到一张大图里面。

（2）Lazyload：图片懒加载。原理：初始指向一个空或者一个小图片，到可视区域再加载。将页面里所有 img 属性 src 属性用 data-xx 代替，当页面滚动直至此图片出现在可视区域时，用 js 取到该图片的 data-xx 的值赋给 src。用 sroll 监听事件。echo.js（实现 lazyload 的库），调用 echo.init()，在里面设置 offset（离可视区域多少像素的图片可以被加载）和 throttle（图片延时加载的秒数）。

（3）利用浏览器缓存

（4）在工程化里面，将写死的图片放在项目中

3.精简资源

（1）css 文件放在 head 里面，js 放在 body 尾部。

（2）压缩 css 和 js，smaller

（3）图片格式:webp

（4）图片压缩，tinypng

4.加快网络

（1）使用 cdn 加速

内容分发网络，实时的根据流量和个节点的情况（负载、到用户的距离）等等，将用户的请求重定向到离用户最近的节点上。

（2）预加载 dns

<link = “dns-profetch” href = “//example.com”>

域名系统，访问目标结点时，必须先将域名转化为 ip 地址。在 chrom 浏览器中，页面中已经有的 url 会被自动地预加载，所以，dns 一般用来预加载可能会用到的 url。dns 预解析是指把 href 后面的转成 ip。因为在主流浏览器中，默认对与页面上出现的 url 或者和这个 url 同域的进行预解析，而不同域的则无法预解析，所以要手动添加。

（3）减少 cookie

cookie 影响加载速度，静态域名不使用 cookie

5.预加载和懒加载

预加载：提前加载相关的资源，存到缓存，之后要使用直接拿即可

懒加载：滚动到视图内时才进行请求 url

6.图片懒加载原理

根据：页面中的 img 如果没有 src 属性或者 src 为空，那么不会发请求

原理：src 设为空，把图片的 url 放到 data-src 属性中（不用在代码中额外保存 data-src），计算到图片将要进入可视区域时，将 url 取出来放到 src 中（el.src）。

计算进入可视区域方法：

1.a 获取可视区域高度（document.innerHeight）->b 获取图片相对于文档顶部的距离（el.offsetTop）->c 获取可视窗口顶部和文档顶部的距离（document.body.scrollTop）->计算 b-(a+c)，小于 0 则进入可视区域

2.a 获取元素与可视区域的左上角的距离（getBoundingClientRect().DOMRect，返回 left、top 等）->b 获取可视区域的高度(document.innerHeight)->计算 a-b

3.IntersectionObserver(callback,option):交叉观察器。交叉指的是可数区域与元素之间的部分,callback 是可见性变化时的回调。这是一个构造函数，需要实例化。通过实例.observe(el)开启观察，通过实例.unobserve(el)停止观察，通过实例.disconnect()关闭观察器。

7.CSS 优化

（1）避免使用 CSS 表达式，css 表达式式用来动态设置 CSS 的，在 CSS 中用 js，是危险的操作，可能极度消耗性能

（2）style 避免直接写在 HTML 中，会阻塞页面渲染。

（3）移除空的 CSS 规则

（4）不滥用 float，float 在渲染时计算量比较大，尽量减少使用

（5）最多三层，使用 less 和 sass 的时候

8.JS 优化

（1）资源异步加载

JS 是单线程的，不管时同步还是异步，同时执行的只有一种操作。异步加载，又叫非阻塞加载，浏览器在执行 js 的同时，还会继续对后续页面的处理。通俗的说，就是打乱了原本单线程中流程处理的先后顺序。

动态脚本加载：JS 里面 innerhtml 等。

defer：H5<script defer>，在 HTML 解析完才会执行，如果是多个，则按照加载的顺序执行。

async：H5<script async>，加载完之后立即执行，如果是多个，执行顺序和加载顺序无关，哪个先解析完返回哪个先执行。（下载完就执行，）

async/await：async 用来处理异步操作，返回一个 promise 对象。await 用来取代 then（解决了 promise 中参数传递不方便的问题，以及优化 promise 的写法）。异步终极解决方案。

Promise:将异步操作封装起来，用统一的方式来使用。

（2）选择器使用

id 选择器和 class 选择器的效率是最高的，尽量使用；伪类选择器的效率最低，通配符也低。

（3）尽量使用事件代理。

9.移动端优化方案

原则：i.PC 端的优化方案在移动端依旧可以使用。

ii.三秒完成渲染或者使用 loading

iii.首屏资源不应超过 1014KB，因为 3G 网速为 338KB/S

iv.移动端因为手机配置的问题，除加载速度外还要考虑渲染速度

v.加载完成之后用户交互使用时也需要注意性能

（1）touch 事件优化，使用 touchestart、touchend 代替 click，响应速度更快。

（2）注重缓存。

10.base64 的缺点

1.图片尺寸变大，但是可以通过 gzip 来压缩，所以影响不大。

2.无法缓存，继续访问还要下载

webpack 中用 url-loader 来加载的话，小于 10k 就会被转码成 base64

四.ES

12.数据类型

1.js 中有哪些基础的数据类型（6 种 ES5+1 种 ES6）

usbnn:undefined,string,boolean,null,number,object

symbol（ES6）

2.symbol 用来干什么的，使用场景

在 js 中，对象的键一般都是字符串，这样往往会重名，导致重写。因此我们用 symbol，symbol 类型的值都是唯一的。

var mysymbol = Symbol()；

这里前面没有 new，因为它是基本数据类型，而不是对象数据类型。

3.undefined 与 null 的区别

undefined: Number(undefined) == NaN

（1）变量被申明了，但没有赋值。

（2）函数没有返回值时，默认返回的时 undefined。

（3）调用函数时，应该提供的参数没有提供，该参数等于 undefined。

（4）对象中没有赋值的属性。

null：Number(null) == 0

（1）作为函数的参数，表示该函数的参数不是对象

（2）作为对象原型链的终点。

4.js 中有哪些对象

（1）Math：算术任务

abs（）: 返回绝对值

ceil（）：向上取整

floor（）：向下取整

max（x1,x2,x3）：取最大值

pow(x,y) y 个 x 相乘

round（）四舍五入

random（）0-1 随机数（0 可以取到，1 取不到）

（2）Date：日期和时间

date.getTime()//返回 1970 年 1.1 到 date 对象所设时间的毫秒数。

（3）Array：数组对象

（4）String：字符串对象

5.js 中的隐式转换

（1）对象和布尔值进行比较==

对象->字符串->数字

布尔值->数字

（2）对象与字符串进行比较==

对象->字符串

[1,2,3] == ‘1,2,3’ //true

（3）Number(null) == 0；Number(undefined) == 0;

链条：对象->字符串->数字<-布尔值

（4）null == undefined

null 和 undefined 和其他任何值比较的时候都不会转化类型，都是 false

（5）加号相当于 concat（如果有字符串的话），其他的符号则转化为数字进行计算

1+’1’ = ‘11’；’1’ + 1=’11’；1-’1’ = 0；1/’1’ = 1

6.判断数组的方法

（1）arr instanceof Array;判断是否为 true（ES3）

（2）arr.constructor === Array;判断是否为 true（ES3）

（3）Object.prototype.toString.call(arr) === ‘[object Array]’（ES3）

（4）Array.isArray(arr)；（ES5,IE9+）

7.instanceof 和 typeof 的不同

typeof 一般用来判断一个变量是否存在，因为对于 Null、Array 返回的是 object。

instanceof 用来判断这个实例对象是否在这个构造函数的原型对象的原型链上。

typeof(“要判断的”)，可以是 undefined、number、string、boolean、object（null，array，object）、function（函数）

8.数组的方法

数组方法	作用	返回值	改变原数组
a.join(”)	数组->字符串，用括号中的连接	字符串	否
a.toString()	数组->字符串，用逗号连接	字符串	否
a.push()	在数组末尾添加元素	添加的元素	是
a.pop()	在数组末尾删除一个元素	删除的元素	是
a.shift()	数组头部删除一个元素	删除的元素	是
a.unshift()	数组头部添加一个元素	添加的元素	是
a.slice()	截取数组,从第一个参数到第二个参数，如果没有第二个参数，默认为末尾	截取的部分数组	否

数组方法	作用	返回值	改变原数组
a.splice(1,5,1,2)	替换数组，从第一个参数开始删除第二个元素的个数，然后加入后面的参数	被替换的部分	是
a.reverse()	翻转数组	翻转后的数组	是
a.sort()	数组排序(按字符串规则)	排序后的数组	是
a.concat(b)	连接数组	连接后的新数组	否
a.indexOf(‘’)	从前往后查找第一个匹配的值	查找到的值的下标	否
a.laseIndexOf(‘’)	从后往前查找第一个匹配的值	查找到的值的下标	否
a.includes(1,1)(ES7)	从第二个参数下标开始检查是否含有第一个元素值	true or false	否
a.filter(item,index,function)	将 item 放入后面的函数中过滤，若返回值为 false，则过滤掉那一项	新数组	否
a.foreach()	对每一项进行操作，替代 for	无	是
a.map()	对每一项进行操作	新数组	否
a.every()	检查是否全部满足	ture or false	否
a.some()	检查是否有一项以上满足	true or false	否
a.reduce(function,flag)	归并方法,接受一个 function(前一个值，当前值，当前下标，数组对象)和一个可选参数	function 里面 return 的最终的值	否
a.reduceRight()	归并方法。从后往前	function 里面 return 的最终的值	否
a.concat(b)	拼接数组	拼接的新数组	否

7.String 的方法

字符串方法	作用	返回	是否改变原字符串
b.concat(c)	拼接字符串	拼接的新字符串	否
b.slice()	字符串截取	新字符串	否
b.indexOf(item,index)	查找字符串，从前往后	第一个找到的下标	否
b.lastIndexOf()	查找字符串，从后往前	第一个找到的下标	否
b.charAt(index)	按下标查找元素	下标返回的元素值	否
b.substring(start,end)	截取字符串	截取的字符串	否
b.split(":")	分割字符串为数组	新数组	否
b.trim()	去除两端空格	新字符串	否
b.trimLeft()/right	去除左边或者右边空格	新字符串	否
reg.test(b)	判断是否包含正则表达式	true or false	否
reg.exec(b)	检索包含的正则	一个结果数组或者 null	否
b.repalce(sds,str)	替换匹配的正则	匹配后的新数组	否
b.match(reg)	检索包含的正则	一个结果数组或者 null	否

8.forEach 与 for in，for of，map 的区别

都是可以用来遍历数组的

for in 除了打印出数组的值外(index.item)，还会将数组的私有属性打印出来，所以不常用。如果对象

，只有 item，打印出来的是 key。

for of 只能用在数组中。没有 index。

foreach 只能用于数组， 不能打印私有属性，会修改原数组。没有返回值

map 只能用于数组，不会修改数组，返回新数组，需要在每个 map 里面 return

9.深拷贝和浅拷贝

赋值：两个对象经过拷贝，有相同的属性，对象也相同（指向同一内存）（直接=即可）

浅拷贝：属性相同，对象不同（不同内存），但是其内部引用类型数据指向的还是相同的内存地址（例如 array 等）。

深拷贝：属性相同，对象不同，内部引用类型数据地址不同。

10.浅拷贝的方法

（1）a = b.slice(0);数组

（2）a = b.concat();数组或字符串

（3）a = Object.assign([],b)；对象即可。将所有可枚举属性的值从一个或多个源对象复制到目标对象，他将返回目标对象。

11.深拷贝的方法

（1）JSON.parse(JSON.stringify(oldObj));将对象转化为 json 字符串，再转化为 JSON 对象，完成深拷贝。（缺点：对象里的函数无法拷贝下来，对象原型链上的属性和方法无法拷贝下来）

（2）函数方法递归拷贝

遍历被拷贝对象（for in）-->判断被拷贝对象的数据类型-->如果是引用数据类型则再次调用，如果不是则直接赋值。

（3）$.extend([deep],target,oldobject)

第一个参数是是否深拷贝，默认为 false，即浅拷贝。

第二个参数为拷贝目标。

第三个参数为拷贝的源目标。

13.二进制和十进制互转

二进制的 string 转成十进制的 number：parseInt(str,2)

十进制的 number 转成二进制的 string: num.toString(2);

14.arguments 对象与数组有什么不同，哪些数组中的方法不能用在 arguments 中。

arguments 的数据类型是一个对象。

arguments 只能用 length 和[0]，其他的数组的方法都是不可用的。

用 Array.prototype.slice.call(arguments,0);可以将其转化为真的数组。

15.arr.sort()

数组的 sort 方法，默认是将数组按照 unicode 编码的。

如果 compareFunction(a,b)函数重写了，那么返回的结果如果是正数的话，a 就在 b 的后面，如果返回的结果是负数的话，那么 a 在 b 的前面。

这个方法可以用来给数组排序。

```
//数组中的数字排序,从小到大
function compareFunction(a,b){
    return a-b;
}
console.log(arr.sort(compareFunction));
//随机打乱数组
function compareFunction(a,b){
    var random = Math.random();
    return random-0.5;
}
console.log(arr.sort(compareFunction));
```

16.解析 url

```
function parseQueryString(url){
    var str = url.split("?")[1],     //通过?得到一个数组,取?后面的参数
        items = str.split("&");     //分割成数组
    var arr,name,value;

    for(var i=0; i<items.length; i++){
```

```
        arr = items[i].split("=");     //["key0", "0"]
        name = arr[0];
        value = arr[1];
        this[name] = value;
    }
}
var url = "http://www.taobao.com/index.php?key0=0&key1=1&key2=2";

var obj = new parseQueryString(url);
console.log(obj);
```

17.深度遍历 dom

```
function deepSearch(node){
    console.log(node.tagName)
    if(node.children.length){
        Array.from(node.children).forEach( function(el){
            deepSearch(el)
        })
    }
}
```

body[]

18.~~与+与 Number

~~后面的 undefined 和 null 都是 0，小数点后面的自动删掉
+就是 Number()，小数部分保留
运算级别都是最后的

13.内存

1.什么是垃圾回收机制

找出不再使用的变量，然后释放掉其占用的内存，这个过程不是实时的，因为其开销比较大，所以会按照固定的时间间隔周期进行。

2.js 中的垃圾回收策略

标记清除：（变量申明时，将变量标记“进入环境”，函数执行结束时，标记为“离开环境”。）垃圾回收器会给内存中的变量标记，过滤掉环境中的变量和环境中的变量所引用的变量，其他的“离开环境”的变量就是要回收的。（大部分浏览器都是这种方式，区别在于如何标记以及垃圾回收间隔，例如特殊位的反转维护一个列表等）

引用计数：跟踪记录每个值的引用次数，如果值被变量引用则+1，这个变量改为引用另一个值则-1。回收那些为 0 的值的内存空间。（低版本的 IE 中使用的，经常会造成内存泄漏）

3.什么时候触发垃圾回收

Ie6 根据内存分配量运行的，环境中存在 256 个变量、4096 个对象、64k 字符串任意一种则会触发。缺点：如果脚本很复杂的话，那么垃圾回收机制会一直在工作。

Ie7+，动态的。占比方式。

4.什么是内存泄漏

指一块被分配的内存不能使用且不能被回收，直到浏览器进程结束。（不再用到的内存，没有及时释放）

<u>5.什么时候产生内存泄漏及解决</u>

（1）意外的全局变量引起的内存泄漏，因为正常模式下，变量没有定义就赋值会默认为全局变量
原因：全局变量不能被回收
解决：使用严格模式避免

（2）闭包引起的
原因：闭包可以维持函数内局部变量。
解决：将事件函数定义在外部，解除闭包。或者再外层作用域内部底部使用变量=null

（3）页面中的元素被移除或者替换时，若绑定的事件没有被移除，则会导致内存泄漏。

解决：手动删除

（4）被遗忘的定时器或者回调

原因：定时器中有 dom 的引用，即使 dom 删除了，但是定时器还在，所以内存中还是有这个 dom 的。

解决：手动删除定时器和 dom

（5）IE 中 onclick 是不能自动回收内存的，所以需要再 onclick 的绑定事件里面，加上 onclick = null。

6.GC 优化策略

（1）分代回收

将对象区分为临时对象和持久对象，多回收临时对象区，少回收持久对象区。

（2）增量 GC

每次处理一点，下次再处理一点。

14.作用域链与闭包

1.什么是作用域链

当代码在一个环境中执行时，会创建变量对象的一个作用域链。作用域链的作用是保证对执行环境有权访问的所有变量和函数的有序访问。整个作用域链的本质是一个指向变相对象的指针列表。作用域链的最前端，始终是当前正在执行的代码所在环境的变量对象。

2.回调函数

回调函数作为参数传递给另一个函数，如果被回调的函数是同步的话，那么一次执行。异步则挂起执行。注意，被回调的函数里面对参数进行的任何操作都不影响父函数的值。

3.申明提升

（1）变量提升，不管变量在哪，都将变量的申明提升到域内的首部。但是没有初始化。

（2）函数提升，将申明和内容都提升到域内的首部。

4.闭包

闭包就是 function that return function

跨作用域访问变量，就是闭包。

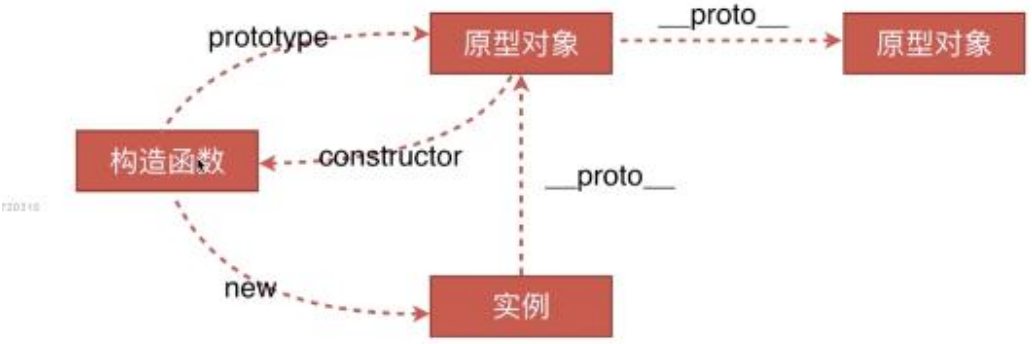
5.闭包的作用

（1）通过闭包可以把局部变量传递出来。

（2）避免污染，相当于构建私有的区域。

15.原型链

1.说一下原型链是什么



构造函数：一个特殊的函数，默认首字母大写，实际上任何的函数都可以作为构造函数，构造函数一般不会写 return。

原型对象：无论何时，只要创建了一个新函数，就会根据特定的规则创建一个 prototype 属性，这个属性指向函数的原型对象。默认情况下，初始的原型对象会自动获得一个 constructor 属性，还有一些从 Object 继承来的方法。实例通过__proto__可以找到原型对象。原型对象也可以通过原型对象.isPrototypeof(实例)来判断原型对象是否是实例的原型。

查找属性：要读取某个对象的某个属性时，都会执行一次搜索，目标时给定名字的属性。搜素先从

实例本身开始，如果没找到，则取找指针指向的原型对象，如没有，则再去上一层上找。

2.in 与 hasOwnProperty()

属性 in 对象：表示这个属性能够通过这个对象访问到，包括对象的原型链上。

hasOwnProperty(对象，属性)：表示这个对象中是不是含有这个属性。

可以结合这两种，来判断属性是否在对象的原型链上。

3.new 发生了什么

（1）创造一个新的实例对象，继承自构造函数指向的原型对象

（2）执行构造函数，将上下文指定给这个新的对象（this）

（3）判断构造函数是否返回一个对象，如果不是，则返回前面的对象，如果是，那么返回构造函数返回的对象。

不管怎样，new 中的 this 都是指向第一步生成的实例对象

4.如何实现继承

(1)利用构造函数

```
function P1(name){
    this.name = name;
}
function P2(name,age){
    P1.call(this,name);
    this.age = age;
}
var person = new P2("leerle",12);
```

缺点：第一个构造函数的原型链上的属性无法继承到

(2)利用原型链

```
function P1(){
    this.name = "leerle"
}
function P2(){
    this.age = 12;
}
P2.prototype = new P1();
var person = new P2();
```

缺点：1.重写子类的原型等于父类的实例，如果父类中包含引用类型，那么会共享。

2.创建子类实例时，无法向父类构造函数传数据

(3)构造函数+原型链

```
function P1(){
    this.name = 'leerle';
}
function P2(){
    p1.call(this);
    this.age = 12;
}
P1.prototype = P2.prototype;
var person = new P1()
```

将引用类型的定义在构造函数里面，避免共享。

(4)ES6 里用 class,extents 将原型链的方法封装起来，更简洁优雅。其实就是（3）的方法

5.创建对象的方式

（1）创建 object，然后添加属性

（2）字面量

（3）工厂模式

用函数封装（1）中的方法，在这个方法中创建对象，并返回一个对象。

（4）构造函数

```
function Person(){
    this.name = 'leerle';
    this.call = function(){
        alert(this.name);
    }
}
var person = new Person();
```

缺点：如果构造函数中有函数，那么每个实例中都要新建这个函数，也就是说不同实例中的函

数是不相等的，这会带来资源浪费。可以定义全局函数解决，但是定义太多的全局函数会影响封装性。

（5）Object.creat

```
var P = {};//作为原型对象。
var o = Object.creat(P);//将 P 当成 o 的原型对象。
缺点：i 取不到构造函数里面的属性
ii 原型对象中如果有引用类型的，例如数组，那么在不同的实例中，这个数组是共享的。
```

（6）原型模式

```
function Person(){
}
Person.prototype = {
        constructor: Person,
        name = 'leerle'
}
var person = new Person()
```

（7）混合模式

```
function Person(name){
        this.name = name
}
Person.prototype = {
        constructor: Person,
        name = 'leerle'
}
var person = new Person('le')
```

16. 函数与箭头函数

1.js 中创建函数的方法

（1）函数声明 fucntion sum(num1,num2){return num1+num2}

（2）函数表达式 var sum = function(num1,num2)(return num1+num2)

（3）用来理解万物皆对象 var sum = new Function("num1","num2","return num1+num2")

2.函数声明和函数表达式的不同

函数声明的函数，其他语句不管是在函数声明之前还是函数声明之后，都是可以调用这个函数的。因为在代码执行之前，解释器会先读取函数的声明，这就是函数声明提升。

函数表达式，就是创建一个匿名函数，再将这个匿名函数复制给一个变量。不能放在调用语句之后。

3.箭头函数的特点

var sum = (num1,num2) => num1 + num2

（1）this 的指向是根据词法作用域来确定的，也就是从里层一直向外找，直到找到或者到 windows 位置。而传统的函数里面的 this 是根据执行的时候的对象来决定的。（this 是借的）

（2）简化了函数的写法。

（3）不能通过 new 关键字调用，也就是说不能作为构造函数，因为没有 this。

（4）没有原型对象，因为不可以通过 new，没有创建原型的需求。

4.立即执行函数

作用是隔离作用域。比如在循环里面创建函数，循环里面给一个外部的 fuc 对象 push 一个关于 i 的函数，由于循环里面的每次迭代都是共享同样一个变量，因此，最后在 fuc 里面的 i 都是一样的，值等于最后一个 i 的值。使用立即执行函数可以强制生成副本，将 i 存为新的量从而保存起来。

5.立即执行函数要注意的地方

不能有 function 开头，因为 js 引擎会识别为函数申明，并且会报错。

所以可以在前面加上!、+、-等符号，最好是使用（）将整个包起来，因为+等还会和函数的返回值进行运算。

17.this 的指向

1.正常情况下 this 的指向

this 指的是运行环境，所以谁调用就是谁。如果没有的话，那么就是 window。在严格模式下，指向

的是

2.箭头函数中

箭头函数中没有 this 绑定，也没有 arguments 等，这些都需要查找作用域来决定其值。如果箭头函数被一个非箭头函数包含，那么 this 就是这个非箭头函数的 this，如果没有的话 this 就是全局对象，也就是 window。（用 forEach 等不是函数）

3.闭包中

闭包中的 this 指向的是 window。不管是不是前面加了对象，因为对于闭包来说，返回的是一个函数，前面的 obj.只是修改了第一层函数的作用域，最终还是只能无奈的返回里层的函数。

```
var obj = {
    name: 'obj',
    x: function(){
        var name = 'x'
        return function y(){
            console.log(this.name)
            return this.name
        }
    }
}
obj.x()()
```

4.匿名函数中

匿名函数中 this 指向 window

5.setTimeout 和 setInterval 中

this 指向 window，严格模式下指向 undefined。

这是因为 setTimeout 调用的代码运行在与所在函数完全分离的执行环境上。

所有异步队列中的方法都是在全局作用域中执行的

1.可以使用箭头函数修改 this

2.用 that=this

17.ES6 中的新特性

1.let、var、const

变量提升：js 中，函数及变量的声明都会被提升到函数的最顶端，但是没有初始化。

var 中遵循变量提升，对同一个变量允许多个 var

let 和 const 中都是块级声明（块级作用域指的是函数内部或者{}之间的区域）。不遵循变量提升，并且不允许重复申明。对于同一个变量不允许多个 let 进行申明

const 为常量申明，不允许修改绑定，但允许修改值。

const person = {name: “1”} 后面可以 person.name = “2”但是不允许

person = {name: “2”}

其他语言中 const 定义的变量都是不可以改变的。

2.es6 下数组解构

允许这样的写法：var[a,b,c] = [1,2]

//a=1,b=2,undefined

也允许对左边进行赋值

var[a = 1,b = a] = [2]

//a = 2,b = 2

3.map

map：键值对。

var m = new Map([[‘a’,100][‘b’,20][‘c’,10]])

用来替换用两个数组存储的情况，比如，学生名字和学生成绩，用 map 则无论这个数组有多大，查找都是极快的。

m.set(‘d’,80)添加

m.has(‘a’)返回 true，判断是否存在

m.get('a')返回 100，根据键查找值
m.delete('a')删除键为 a 的键值对

4.map 和对象的区别

1.map 中 key 可以是对象，而对象中 key 只能是 string 或者 symbol

2.map.length 可以获取到 map 的长度，而 object 一般使用 Object.keys(obj)返回 key 值集合的数组，然后.length

5.set

set：键的集合
var s = new Set([1,2,3])
和 map 类似，但是只存储了键，没有存储 value

6.ES7 中的新特性

（1）**：指数运算符，等于^，和 Math.pow（）一样。运算符的级别时最高的，另外，x**y 前只能加上++或者--。

（2）Array.prototype.includes()方法:判断数组中是否存在延续相似的数组。

```
var values = [1,2,3,4,5];
console.log(values.includes(1,2))//false
```

第一个参数表示为要查找的值，第二个值表示要查找的起始位置，如果没有则默认为 0.

注意，在 values.indexOf()中，就算数组中有 NAN，那么也是返回-1（表示查不到），因为 NAN===NAN 为 false，但是他们确实被认为是相等的。而在 includes 中则可以找到。

还有一点，在 indexOf 和 includes 中，+0 和-0 和 0 都是相等的，而如果用 Object.is（）则是不相等。

7.async 和 await

用来取代 promise 的异步解决方案，用同步的写法来实现异步。

async 中使用 return ，返回的是 promise 对象，如果使用 await，那么 await 后面的方法或者 promise 会加入运行栈，await 这行代码之后的都相当于异步的（相当于 promise 的 resoled 之后的）。

8.尾调用优化

在函数的最后一项中，如果使用的 return fucntion()的话，那么将会对这个调用进行优化。

函数调用会形成一个调用帧（调用记录），如果 A 函数中调用了 B 函数，那么，在 A 函数的调用帧上，会生成 B 函数的调用帧，（B 帧）这个调用帧里面保存了调用位置和用到的 A 的变量。当 B 执行完，返回结果给 A，A 帧才会消失。

由于尾调用中不需要外层函数的调用帧，因此只保留外层函数的调用帧即可。这就是尾调用优化。

需要满足的条件：1.return 后面的函数中没有用到外层函数的变量。所以递归必须把局部变量写成函数的参数

2.return 后面只能是一个方法，不能有其他的操作。

实例尾递归：斐波那契

```
function fib(n,n1=1,n2=1)
    ...
    return fib(n-1,n2,n2+n1)
```

五.HTML5

18.HTML5

1.HTML5 不同于以前版本的特点

（1）新增了很多标签，支持了 HTML 语义化
（2）对搜索引擎更友好：爬虫依赖于标签来确定上下文和各个关键字的比重
（3）简化了各种标签、属性等等，例如 HTML5 声明<!doctype html>
（4）更多的 api，例如 websokect 实现跨域通信等

2.HTML5 下新的 api

（1）画布 canvas：相比于 SVG，更适合用来做游戏等。
（2）本地存储 localstorage 和 sessionstorage
sessionstorage 存储 value：setItem（key,value）
sessionstorage 获取 value: getItem（key）
（3）websocket 和 postmessage

3.语义化标签有哪些

header：主导航、搜索框等

main：页面主要内容

aside：一般放在 main 里面，包括侧栏、广告等

footer：页脚

section：有内容的一部分，比如网页上的标题、简介等。

article：报纸一样的，包含的是独立的模块，或者引入外部文章等。比如说要引入一个博客，就用 article，如果其中有评论的话，也可以嵌套 article。section 的进阶。

一般用法：

```
article
    --header
        --h1 标题
        --p 发表日期
    --p 文章内容
    --section 多条评论
        --article
        --article
```

4.语义化标签的作用

为了改善以前 div+css 泛滥的情况，让机器和人能够读到符合规范的代码，所以用 html 语义化，将标签的作用规定好。

html 语义化的好处：

1.就算不使用 css，也能呈现良好的内容结构和代码结构。

2.提高代码可读性。提高机器解析的效率。

3.SEO

5.canvas 绘画的基本过程

第一步：创建一块画布，并赋予一个 id

```
<canvas id = “mycanvas” width=”200” height = “200”></canvas>
```

第二步：js 获取 id，获取一个用于画布上绘图的环境

6.ol 和 ul，以及怎么去掉前面的圆点或者数字

ul：无序列表

ol：有序列表 ze）

list-style:none

7.移动端点击穿透问题

场景：当一个弹窗上的点击关闭按钮是用 tap 或者是用 touch 来触发事件，而下一层的是使用 click 来触发事件的，那么当点击关闭之后，弹窗关闭，下面的元素经过 300ms 会触发 click 事件。

为什么：移动端浏览器中 click 中会有 300ms 的延迟才会触发（为了实现双击放大的监听），touch 和 tap（使用 touchstart 和 touchend 来模拟实现的）则是用来解决这个延迟的，但是当触发 tap 的时候，下一层的元素会在 300ms 之后检查 currentTarget 和 target，这时候也会触发 click 的事件

解决方法：1.全部使用 touch，不推荐，因为 pc 上没有 touch 事件

2.全部使用 click，不推荐，因为在移动端会有 300ms 延迟

3.使用 fastclick 等库来解决。原理：在 touch 里面使用 preventDefult 和 stopPropagation，阻止事件到下层元素（将弹窗写成下层元素的子元素）

8.有哪些行内元素，哪些块级元素

行内元素：a，span，i（斜体），em（强调，效果等同于 i）

特殊的行内元素：textarea，img

块级元素：div，p，ul，ol，li

9.ul 中单击 li，输出当前 li 的编号

1.直接给每个 li 绑定事件

```
var list = document.getElementsByTagName('li')
for(var i =0; i<list.length;i++){
    list[i].index=i
    list[i].onclick=function(){
        console.log(this.index)
    }
}
```

2.利用事件委托

```
var ul = document.getElementById('list');
```

```
var li = ul.getElementsByTagName('li');
ul.onclick = function(e){
	var target = e.target
	var re = 0
	for(var l in li){
		if(target == l){
			console.log(re)
			return
		}
		re++
	}
}
```

10.函数防抖和函数节流

场景：列表滚动触发事件的时候，事件出发的频率非常高，会导致滚动不流畅，所以需要把 js 代码的执行次数控制在合理的范围。防止出现卡顿。（函数节流）

函数防抖：多个相同的事件只执行一次，并且是在 end 之后一段时间触发。思路是设定一个定时器，如果触发事件就会重置定时器，直到 300ms 没有触发定时器，才会触发事件。

函数节流：每隔多少秒才会触发这个事件

11.函数科里化

科里化表示可以使用 add(1)(2)的方式实现 1+2

实现原理：

```
function add(x){
  return function (y){
    return x + y
  }
}
//add(x)(y)
```

12.webworker

相当于在 js 里面实现多线程，运行在浏览器后台，不影响页面的性能，但是比较耗费资源，所以一旦使用完毕，应该立即关闭。

原理：在当前的 js 主线程中，使用 Worker 类加载一个 js 开辟一个新的线程，起到互不阻塞的效果。通过 postMessage 和 pnmessgae 来进行通信。可以用 webworker 来处理一些复杂的比较耗时的计算，例如轮询服务器状态等

缺点：1.同源限制

2.无法读取到主线程所在页面的 dom 对象

3.无法读取本地文件，只能读取网络文件

4.无法 alter 等方法，但能 ajax

5.浏览器兼容性不一样

六.CSS3

6.1 盒模型

1.盒子模型（标准模型和 Ie 模型的区别）

标准：宽和高不计 border 和 padding

2.CSS 怎么设置两种模型

box-sizing = content-box

box-sizing = border-box

3.怎么获取盒模型对应的宽和高

dom.style.width/height//只能改内联样式中的

dom.currentStyle.width/height//渲染中的，只能 ie

windows.getComputedStyle.width//通用

dom.getBoundingClientRect().width/height

6.2 选择器

1.常见选择器

（1）标签选择器
（2）类选择器（.xx）
（3）ID 选择器（#xx）
（4）全局选择器（*）
（5）组合选择器（xx，yy）
（6）兄弟选择器（XX+YY）
（7）子元素选择器（XX>YY）只作为 XX 子元素的 YY，也就是 XX 为 YY 的直接父元素
（8）后代选择器（XX YY）

2.选择权重及权重计算规则

内联样式表 1000
ID 选择器 100
Class 选择器 10
标签选择器 1
其他的都为 0（包括继承）
从高到底的顺序，如果全都一样，那么就会覆盖相同的属性。

3.伪类与伪元素的种类

在 css3 中，伪类用:伪元素用::。（对于 css3 之前已经有的伪元素，也可以用:）

伪类：active、hover、nth-child(n)、first-child、nth-last-child、last-child、last-of-type、target（锚点）、checked（只能用在 input 的 type 为 checkbox 或者 adius，单选 or 多选）

伪元素：after、before、first-line（第一行）、first-letter（第一个字母或汉字）

4.伪类和伪元素的区别

区别：1.最主要的区别在于是否抽象创造了新元素。例如使用 first-letter 就是抽象创造了一个新的 span，不用伪元素的话必须要创建新的 span。

2.伪类就和真正的类一样，可以无限叠加，而伪元素在一个选择中只能出现一次，并且要在最末尾

3.计算权重时，伪类=类，伪元素=标签

5.选择 ul 中倒数第一个 li

li:last-child 可以用来查找最后一个元素，当这个元素是 li 时

如果 li 后面还有其他的元素，那么用 li：last-of-type，可以找到最后一个 li

6.3 flex+position+float

1.flex 属性

flex-wrap：nowrap（默认不换行）|wrap|wrap-reverse

flex-direction：row（默认从左到右）|row-reverse|column

justify-content：flex-start（左对齐）|flex-end（右对齐）|center（居中）|space-between（两端对齐，中间居中）|space-around（两个项目中间的间隔相等）

align-items：flex-start（上端对齐）|flex-end（下端对齐）|center（中间对齐）|baseline（项目的第一行文字的下线对齐）|stretch（默认值，占满容器）

order：默认为 0，数值越靠前排的越前

2.flex 属性之后的三个值是什么含义

flex：flex-grow flex-shrink flex-basis|auto。。。（简写 flex：1）

flex-grow 如果元素加起来不够父元素的大小，flex-grow 用来扩充，默认为 0，即为子元素原本的大小，不进行扩大。flex-grow 越大，子元素越大

flex-shrink 如果元素加起来超过父元素的大小，flex-shrink 用来缩放，可选，默认为 1。当超过，那么将超出的 px 按照各个子元素的 flex-shrink 的进行缩放。flex-shrink 越大，子元素越小

flex-basis 设置元素的基础长度（轴上的），默认值为 auto 即元素本来的长度，也可以设置 px

3.position 有哪些属性

absolute:绝当定位，相对于最近一级的定位不是 static 的父级元素进行定位；

fixed（老 ie 不支持）：生成绝对定位的元素，通常相对于浏览器窗口进行定位；特殊版的 absolute

relative：生成相对定位的元素，相对于其在普通流中的位置进行定位；

static：默认值，没有定位。

sticky：粘性定位，在屏幕范围（viewport）范围之类，是 static，而超出了屏幕范围之后则变成了

fixed

initial：表示元素为他本来的样式，不会被后设置的样式而改变。例如父元素设置了一个属性，而子元素设置了 inital，则父元素的样式不在在子元素中生效。

inherit：规定从父元素继承 position 属性的值。

unset：如果有继承，相当于 inherit；如果没有继承，相当于 initial

4.什么是相对定位和绝对定位，详细说说

相对定位：position:relative，依旧在正常的文档流里面，使用的 top、left 等属性只是相对于他原本的位置而言的，所以他会覆盖其他的元素。

决定定位：position:relative，脱离了文档流，仿佛其他元素不存在一样。相对的位置是最近的已定位祖先元素。

5.子 absolute 父 relative，那么子元素是根据父元素的什么定位的

父元素的 border 的

6.float 注意的地方

float 如果遇到文档流里面的元素，那么就会换行

6.4

4.px，em 和 rem

px：屏幕设备物理上能够显示出的最小的点（每台设备不同），一个 px 的点不一定是正方形。

em：浏览器默认字体尺寸是 16px，1em=16px。但是如果父元素里面的 font-size 改了，则会重新计算。所以不适用。（这里指的是相当于使用那条属性的元素的父元素，比如在父元素里面调用了 font-indent：2em，那么子元素的缩进其实是相当于这个父元素的）

rem：font-size 不会继承父元素的，只会根据 html 里面的 font-size。

5.移动端开发中适配各种屏幕尺寸的解决方案（至少 3 种）

（1）
```
<meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-scale=0.5, maximum-scale=2.0, user-scalable=yes" />
```

i.允许网页宽度自适应调整在网页代码的头部，加上一行 viewport 元标签。

viewport：网页默认的宽度和高度，定义了一个虚拟的 layout viewport。dip（设备逻辑像素，如果网页没有缩放，那么一般等于一个 css 像素等于一个 dip）

initial-scale=1.0：原始缩放比例为 1.0，即网页初始大小占满屏幕面积的 100%。布局 viewport=理想视口。

maximum-scale：允许用户所放的最大比例。

minimum-scale：允许用户所放的最小比例。通常不用

user-scalable：是否允许用户手动缩放。

ii.不使用绝对宽度

width：百分比

iii.字体改为相对大小 em

iv.流式布局。每个区块的位置都设定为浮动，width 为百分比，float 的好处是如果宽度太小，放不下两个元素，后面的元素会自动滚动到前面元素的下方，不会出现滚动条。

v.@media 规则，同一个 css 文件中，根据不同的屏幕分辨率，选择不同的 css 规则。

vi.图片自适应，实现图片的自动缩放，max：width：100%。也可以用媒体查询，然后放上不同大小的图。

（2）网页缩放比例的方式，以最小的屏幕做一版数据出来，然后通过 js 去控制 viewport 的 initial-scale，网页缩放比例。在<meta>标签里面。

缺点：因为 initial-scale 越来越大，会使得网页拉伸的越来越大，导致在大屏幕的移动端下，文字图片会变得模糊。

（3）rem 方法：相对于根元素的字体大小的单位。

缺点：会影响设计的初衷和美观，如果是在大屏幕上设计好了，然后到小屏幕上看，那么文字会变得很小。

6.CSS 隐藏元素的方法以及其区别。

（1）visibility:hidden.。。。。设置为 visible 则为可见

元素被隐藏，但是元素占据的空间依然存在。

（2）opacity：0

元素变透明，空间依然存在。支持 transition:属性 2s，通常用来制作元素的淡入淡出效果

（3）position：absolute。left 或者 right 设置很大的负值。

使元素脱离文档流，不占空间

（4）dispaly：none

不占空间，通常用来做手风琴。

7.怎么在不引入图片的情况下，引入 icon

（1）使用 font-awesome 代替 icon 小图片。引入 font-awesome.css 文件，然后直接定义 class 就行了。

（2）bootstrap 中的 glyphicons，200 多种图片，也是直接定义 class 就行。

（3）用 border 自己画，例如三角形

8.CSS3 有哪些新的属性。

（1）flex 布局

justify-content 子元素水平对齐

align-items 子元素垂直对齐

（2）CSS3 动画，@keyframes 规则

（3）渐变，transition

（4）盒模型新增了一些方法，resize：指定一个元素是否应该由用户去调整大小；box-sizing 允许更改哪一种盒模型

（5）多媒体查询。（5 种不同的）

9.盒子边界

父元素和子元素都是绝对定位的，子元素设置为 top：30px，那么子元素的定位是相对于父元素的 margin 的外边缘来定位的。

10.CSS 中哪些属性是可以被继承的

1.字体系列的属性：

font-family：字体

font-weight：字体粗细

font-size：字体尺寸

2.文本系列属性

text-align：文本水平对齐

line-height：行高

color：文本颜色

text-indent：文本缩进

3.元素可见性

visibility

4.光标属性

cursor：pointer

所有元素：visibility（透明），cursor（光标）

内联元素：字体系列属性（font），除了 text-align、text-indent 之外的文本属性

块级元素：text-align、text-indent

注意：如果是 inline-block，看作是块级元素

举例：父元素使用了 text-indent，子元素如果是块级的话，会继承这个属性，导致后面的人修改代码的时候难以理解。

11.block、inline、inline-block 的区别

block：元素独占一行，block 元素高度自动填满其父级元素，可以设置 height 和 width，可以设置 margin 和 padding

inline：不会独占一行，width、height 不产生效果，margin、padding 只在水平方向上有效

inline-block：不会独占一行，其他的和 block 一样。

12.@import 和 link 的区别

（1）link 是 html 标签，在 html 文件中引入，@import 在 css 文件中引入

（2）link 没有兼容问题，@import 需要在 IE5 以上才生效

（3）link 中的会直接加载，而@import 的会在页面全部被下载完才会被加载

（4）dom 控制样式支持 link，不能修改@import

13.为什么图片等 inline 的可以设置宽高

因为他们拥有内在尺寸，内在尺寸可以设置宽高等属性，等同于 inline-block。

14.calc

作用：动态计算长度值，支持四则元算。

width: calc(100% - 50px):百分百长度向左减少 50px。

15.vertical-align

作用：让子元素根据父元素的基点对齐。垂直对齐。

什么时候生效：如果父元素不是 inline，那么必须设置行高（line-height），子元素必须不是块级元

素（例如图片相对于父元素中的文字对齐）。

值：baseline: 元素放在父元素的基线上，默认(英文的下边缘)

text-top:子元素的顶端与父元素的字体的顶端对齐（中文的上边缘）

text-bottom：子元素的底部与父元素的字体的底部对齐（中文的下边缘）

middle：父元素的中间

17.js 修改 css

（1）`el.style.background = "red";`

（2）`el.style.cssText = "background:red";`

（3）改标签上的 class、id、type、href 等 `el.setAttribute("id", "child");`

18.margin 为负值时

margin-top 为负值的时候，会向上位移

margin-bottom 为负值的时候，不会位移，但是会减少高度

19.选择器解析顺序

从右往左，从里层到外层。

这样在生成渲染树的时候，查找 cssom 的时候，不需要遍历整个 cssom，如果是从左到右的话，为了防止丢掉某些公用的样式，每次都会对整个 cssom 进行一次查找。而如果是从里层到外层的话，那么每次只要查找一部分即可。

另外最多三层选择器嵌套，是因为可以减少查找 cssom 的节点的个数。

20.行内元素之间的间距如何解决

1.行内元素写在一行，因为这是因为换行导致的空白

2.使用 margin 负值

3.父元素设置 font-size=0，不能有文字

21.bem 规范

——代表元素不同的状态

__代表子元素

22.避免样式全局污染

1.scoped：实现原理（给 html 上添加 data-v-1312sd，在 css 中选择器后面添加 data-v-1312sd）缺点（在 scoped 组件中不能修改其他的组件的样式）

2.给最外层 div 加上唯一标识

23.scoped 穿透

解决 scoped 无法修改其他组件的样式

使用>>>

20.css 动画

1.实现动画的方式

1.transtion (必须要有事件才能触发，并且只能触发一次)

2.animation @keyframe

3.SVG

4.canvas（使用 setTimeout 和 setInterval）

5.requestAnimationFrame

```
//transition 实现鼠标移到元素上开始旋转一圈
//transform 表示要动画的属性，3s 表示动画的持续时间，linear 表示动画平滑程度（这里是平缓）
，0s 表示动画开始前等待的时间
div{
    width: 10vw;
    height: 10vw;
    background-color: aqua;
    transition: transform 3s linear 0s;
}
div:hover{
```

```
    transform: rotate(360deg);
}

// animation 实现无限旋转的正方形
//infinite 表示无限下去，永远不会触发动画结束事件
div{
    width: 10vw;
    height: 10vw;
    background-color: aqua;
    //my:动画的名字 3s：动画持续时间 linear:动画的平缓程度 infinite：无限
    animation: my 3s linear infinite;
}
@keyframes my {
    0%{
        transform:rotate(0deg);
    }
    100%{
        transform:rotate(360deg);
    }
}

//SVG
//正方形
//fill:填充的颜色 stroke:边框的颜色 stroke-width:边框的宽度
<svg>
    <rect x="50" y="20" width="150" height="150"
          style="fill:blue;stroke:pink;stroke-width:5;"/>
</svg>
//三角形
//points 表示各个角的坐标
<svg>
<polygon points="200,100 100,300 300,300"
            style="fill:lime;stroke:purple;stroke-width:1"/>
</svg>
//五角星
<svg>
    <polygon points="100,10 40,180 190,60 10,60 160,180"
            style="fill:lime;stroke:purple;stroke-width:5;" />
</svg>
//canvas
//画正方形,必须有 html 的元素挂载
<canvas id="mycanvas" width="200" height="200"></canvas>

var el = document.getElementById("mycanvas")
```

```
var context = el.getContext("2d")
    //fillStyle：填充的颜色 fillRect：前两个参数起始点 x、y 坐标，后两个指长和宽
context.fillStyle = "#ff0000"
context.fillRect(10,10,190,190)
//画三角形
var el = document.getElementById("mycanvas")
var ctx = el.getContext("2d")
ctx.beginPath()
//起始点位置
ctx.moveTo(100,0)
ctx.lineTo(50,50)
ctx.lineTo(150,50)
ctx.closePath()
//stroke 会实际的绘制 lineTo 的线条,fill 则为填充
ctx.strokeStyle="green";
ctx.stroke();
//进度条
myDiv.style.width = '0';
cancelAnimationFrame(timer);
timer = requestAnimationFrame(function fn(){
    if(parseInt(myDiv.style.width) < 500){
        myDiv.style.width = parseInt(myDiv.style.width) + 5 + 'px';
        myDiv.innerHTML =     parseInt(myDiv.style.width)/5 + '%';
        timer = requestAnimationFrame(fn);
    }else{
        cancelAnimationFrame(timer);
    }
});
```

2.transition 与 animation 的区别

最大的区别在于 transition 必须要触发事件才会开始动画，而 animation 则没有限制
animation 在 transition 上提供了更多更丰富的 api
他们都有兼容性问题

3.判断 transition 和 annimation 动画结束的事件

transitionend　animationend

5.transform

transform: rotate(30deg); 顺时针旋转 30 度
transform: translate(10px, 20px);向左移动 10px，向 top 移动 20px
transform: scale(2,4); 宽度放大两倍，高度放大 4 倍

6.SVG 与前两种 css 动画的不同

SVG（可伸缩矢量）是 XML 的，放大缩小图片质量不会有损失

7.requestAnimationFrame()

requestAnimationFrame 优于 setTimeout/setInterval 的原因在于，他是浏览器专门为动画提供的 API，浏览器对他进行了优化。

使用 setTimeout 的时候，后面跟着的时间只是在这个时间将这个动画操作加入了任务队列中，如果

任务队列中前面还有其他的任务，就会导致动画流畅度出现问题。

8.canvas 清除画布的方法

1.由于 canvas 中宽度和高度变化时，canvas 就会被重设，所以可以 c.height 改变
2.ctx.clearRect(x,y,c.width,c.height)

9.纯 css 实现 tab 切换

1.使用 target 锚点，href 中使用

html

```
<ul>
    <li class="">
        <a href="#m1">第一个</a>
    </li>
    <li>
        <a href="#m2">第二个</a>
    </li>
</ul>
<div id="m1">第一个第一个第一个</div>
<div id="m2">第二个第二个</div>
```

css

```
ul{
    list-style: none;
    display: flex;
}
#m1{
    display: none;
}
#m2{
    display: none;
}
#m1:target{
    display: block;
}
#m2:target{
    display: block;
}
```

2.使用 input,type 设置为单选框或者是多选框，可以使用伪类:checked 来获取到，使用 label 和 for 将 label 与 checkbox 联系起来

20.BFC，清除浮动

4.什么是边距重叠，边距重叠具体

边框重叠就是两个元素之间的 margin 和大于两个元素 border 的距离，就会发生边距重叠
重叠的原则：取最大值；空元素上面的和下面的取最大值

5.BFC 的基本概念

块级格式话上下文。BFC 主要用来清除浮动的，就是在页面中创建一个独立的容器，容器内部无论怎样都不影响外部。

6.BFC 的原理（渲染规则）

(1)BFC 的垂直方向上的边距会发生重叠
(2)BFC 的区域不会与浮动元素的 box 重叠
(3)BFC 在页面上是一个独立的容器
(4)计算 BFC 的高度时，浮动元素也会参与运算

7.怎么创建 BFC

设置了浮动
position 不是 static 和 relative
display 为 table-cell 之类的
overflow：auto/hiden

8.BFC 的使用场景

BFC 子元素即使是 float，也会参与计算。比如说，一个父元素里面只有有一个 float 的子元素，如果没有设置 BFC，那么父元素的高度还是零，只有将父元素设置为 BFC，才能将 float 的宽度参与到父元素的计算。

float 与一个同行上相邻的元素，当相邻的高度大于 float 的高度时，超出高度的部分会向 float 没有的那部分侵染，很不美观。需要给相邻的元素设置一个 BFC，就不会出现这样的情况了。

9.怎么清除浮动

（1）在父元素添加 overflow:hidden/auto
（2）额外标签
浮动的盒子后面加上一个标签
.clear{ clear:both} .如果是内部标签，那么会将这个浮动盒子的父元素重新撑开，如果是外部元素，会将这个浮动盒子的影响消除，但是不会撑开父元素。
（3）伪元素。
.hezi:after{ clear:both}
.hezi{zoom：1}//考虑 ie，zoom 设置或检索对象的缩放比例。

22.CSS 实现（定位和导航）

1.实现左边固定 300px，右边自适应

（1）绝对定位
爸爸：position：relative;
左边：position：absolute;
left：0；width：300px;
右边：position：absolute;
right：0；left：300px;
（2）float
左边：float：left；wdith：300px；
（3）flex 布局
爸爸：display:flex
左边：width：300px
右边：flex：1
（4）表格布局
爸爸：display:table；width:100%
左边：display：table-cell；width：300px
右边：display：table-cell；
（5）栅格布局
爸爸：display：grid；

grid-template-columns：300px auto

2.实现左边固定 300px，右边固定 300px，中间自适应

（1）绝对定位(必须设置行高)

爸爸：position: relative;

左边：position：absolute;

left：0; width：300px;

中间：position：absolute；left：300px；right：300px

右边：position：absolute；

right：0；width：300px；

（2）float(中间元素可以设置 margin，另外，float 的元素要先写，因为 float 遇到块级元素会自动换行)

左边：float：left；wdith：300px；

右边：float：right；width：300px

（3）flex 布局

爸爸：display:flex

左边：width：300px

中间：flex：1

右边：width：300px

（4）表格布局

爸爸：display:table；width:100%

左边：display：table-cell；width：300px

中间：display: table-cell；

右边：display：table-cell；width：300px

（5）栅格布局

爸爸：display：grid；

grid-template-columns：300px auto 300px

3.文字居中

（1）行内元素垂直居中

设置 line-height 等于父元素的高。

（2）行内元素水平居中

text-align:center

4.块级元素水平-垂直居中

（1）绝对定位

top:50%;transfrom:translateX(-50%);

left:50%;transfrom:translateY(-50%);

（2）flex

父元素设置 display：flex；

margin：auto

（注意如果不设置成 display：flex 的话是不能垂直居中的）

（3）flex

父元素设置：justfy-content：center（水平居中）

父元素设置：align-items：center（垂直居中）

5.画正方形

（1）设置 width=height

（2）自适应正方形。1vw=1%viewport width

width:50%;height:50vw

（3）自适应正方形。padding

width:50%;padding-bottom:50%;

padding 会把盒子撑开

6.实现三层导航（第一层水平，第二层垂直，第三层换行垂直）

第一层：用一个 ul 包裹一些 li，li 里面放 a 标签。ul 属性 list-style:none（去掉前面的点等默认属性），overflow:hidden(清除第一层标签的浮动)，li 属性 float:left。

第二层：在一个 li 里面再加上一层 ul，ul 里面包裹 li（第二层），li 里面放 a 标签。

7.块级元素中多行文本垂直居中

父元素不定行高：设置 padding-top 和 padding-bottom 相等即可

父元素定行高：

```
<div>
  <p>
    撒大苏打实打实大苏打萨达发生发生萨达萨达沙发沙发沙发按时发生飞洒飞洒打算发士大夫撒旦撒打算萨芬撒大撒大撒法撒旦飞洒大沙发
  </p>
</div>
<style>
  div{
    display:table
  }
  p{
    display:table-cell
    vertical-align:middle
  }
</style>
```

七.综合性问题

1.

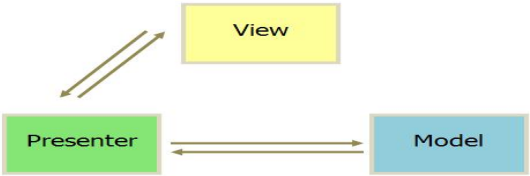
六.工程化

6.1 工程化

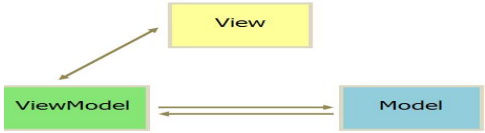
1.MVC、MVVM、MVP

MVC:view（展示）model（数据库）controller（逻辑）view 传送指令到 controller，controller 完成业务逻辑后，要求 model 更改状态，model 将新的数据发送到 view，要求改变状态。单向通信。

MVP：都是双向的，view 非常薄，只做展示，逻辑都在 presenter



MVVM：和 MVP 非常像，唯一的区别是采用双向绑定，view 的变动自动反映在 viewMode。



6.2 webpack

1.webpack 与 gulp 的区别

webpack 是一个前端模块化的方案，更侧重于模块打包。可以把开发中的所有资源（图片、css 文件）都看成模块，通过 loader（加载器）和 plugins（插件）对资源进行处理，打包成符合生产环境部署的前端资源。适合用于单页面开发。

gulp 强调的是前端开发的工作流程，通过配置一系列的 task，定义 task 处理的事务，然后定义执行顺序。侧重的是规范前端开发流程。适合用于多页面开发。

webpack 是在 gulp 处理大型 spa 很麻烦的情况下才出现的。

2.loader 和 plugins 分别用来干嘛的

loader 是在打包构建过程中用来处理源文件的，一次处理一个文件

plugins 是插件，对整个过程起作用的，比如生成 html，压缩 js、css，给 css 加上头部等

3.loader 规范

{

```
    //正则匹配
    test: /\.js$/,
    //loader 集合
    loader:'bable',
    //参数
    option:{}
}
```

每个 loader 就是处理一个模块的，可能有多个 loader 来完成一个模块。

4.常见 loader

1.样式

css-loader：解析 css 代码

style-loader：将 css 模块作为样式导入到 html 中

less-loader：加载和转义 less 文件

sass-loader：加载和转义 sass 文件

2.vue 组件

vue-loader：加载和转义 vue 组件

3.脚本转义

bable-loader：将 es6 转义为 es5，让浏览器能够解析

4.webpack2 和 4 的区别

1.目标零配置：添加了 mode，在 dev 环境下打包时间更快，pro 环境下打包文件体积更小。（学习 parcel 的，在 vue-cli3.0 中使用的是零配置）

2.HtmwebpackPlugins 等插件有改动。

未来的目标：实现缓存和多进程

5.webpack 组织结构

package.json：配置各个依赖包和对外抛出的运行方式

webpack.config.js：配置入口，出口，loader 和 plugins 等

src：放代码的地方

6.webpack 核心概念与配置项

entry：入口

pagen：入口的地址文件路径名字，可以设置多个

output：出口

--filename:打包之后的文件名字

--path:存放地址

--publicPath:配置 cdn（非必需）

loader：规则，类似于 gulp 中的 task，将不同的语言转化为 js（es6.ts），将图像转化为 base64 等

plugins：插件

mode：'production'/'development' ：将 NODE_ENV 设置为两种模式，production 中启用更多的插件

7.webpack 中的模块和依赖图

moduls 模块：将程序拆分成更小的离散功能块，就叫做模块。通过 import 或者@import（css 中）或者 require 等来导入

依赖图：一个文件依赖与另外一个文件，则他们之间有依赖关系，webpack 从入口开始，递归的构建一个依赖图，这个依赖图包含着应用关系所需的各个模块，然后打包为少量的 bundle

8.mainfest 和 runtime

runtime：模块交互时的逻辑，包括模块加载和解析逻辑

mainfest：mainfest 中存放的是提取出来的模块的要点，在 runtime 时通过 mainfest 来查找相关的模块

9.HMR 模块热加载

HMR 会在应用程序运行过程中替换、添加、删除模块，从而无需加载整个模块。通过以下的方式：

1.重新加载时保留之前的状态

2.只更新改变的内容

3.调整样式更加快速

。。。。。

10.webpack 打包优化

通过安装 analyz 来查看打包的文件的体积。一般过大是因为引入的依赖包过大或者是业务代码过于

繁琐。

抽离公共依赖，例如 vuex、vue、jquery 等文件。

1.可以使用 cdn 在 script 中引入，在 webpack.config.js 中使用使用 externals 来告诉 webpack 这些资源从哪里找，然后在 vendor.js 中写入不用打包的文件名。浏览器会进行缓存。

2.使用 dll 方式，动态链接库，就是告诉 webpack 指定库在项目中的位置，从而直接引入，不必打包在内。通过 webpack.dllPlugin 插件来实现

11.webpack 打包的原理

6.3 vue-cli

1.vue-cli 是什么

2.vue-cli3 目录结构

3.vue-cli2 与 3 的区别

6.4 npm 与 yarn

2.npm 与 yarn 的区别

都是包管理工具。

npm 通过语义化控制的思想来设计的。主版本号-次版本号-补丁版本号。～（录入版本之后任意添加的都可以）。^(～再加上向后的所有版本)。所以就算有相同的 packet.json，但是也可能在不同的电脑上安装不同版本的依赖包。npm2 开始可以使用 shrinkwrap 来实现版本控制，但是只会锁定库的版本，而不是库的内容，而 npm 管理员依然可以强制更新。

yarn 有一个 yarn.lock 文件，用来锁定安装的版本。yarn 的速度比 npm 要快，它是采用的并行下载的方式，而 npm 是采用的顺序下载。

他们都是可以缓存下载好的一些包。

七.vue

1 vue 基础

1.vue2 的生命周期，每一个生命周期用来干什么的

step1：通过 new Vue() 创建 vue 实例，初始化事件和生命周期，然后触发 beforeCreated。（没有 dom，没有 data）

step2：挂载数据，绑定事件，然后触发 created。可以获取 data，但是修改 data 不会触发 updated。一般用来获取初始数据。

step3：找实例或者找组件对应的模板，编译为虚拟 dom 放入 render 函数中准备渲染，然后触发 beforeMounted。虚拟 dom 创建完成，修改 data 不会触发 updated。一般用来获取初始数据。

step4：执行 render，渲染出真实的 dom，然后触发 mounted。data 和 dom 都可以获取到，并且可以操作，会触发 updated。

step5：组件或者实例数据更改之后，触发 beforeUpdated。重新构建虚拟 dom，与之前的虚拟 dom 树利用 diff 算法进行对比后更新。

step6：执行 updated，数据全部修改完成，dom 也重新渲染。可以操作更新后的虚拟 dom

step7：beforeDestroy，做善后工作，例如清除计时器等

step8：destroyed，善后工作，去掉数据绑定、监听等，只剩 dom 空壳

2 虚拟 dom，为什么要用虚拟 dom

虚拟 dom 是将真实的 dom 的数据抽取出来，用对象的方式模拟真实的树形结构。每次渲染都会生成新的 vnode 树，通过 diff 方法进行对比，之后修改成新的 vnode 树，然后将新的 vnode 树进行渲染。

3.diff 的作用与比较方式

diff 的作用就是调用 patch 函数，比较新旧节点 vnode 树，便比较边给真实的 dom 打补丁。

比较方式：（仅在同级的 vnode 之间做 diff）逐个遍历 newVdom 的节点，找到它在 oldVdom 中的位置，如果找到了就移动相关的 dom 元素，如果没有找到就说明是新增节点，新建一个节点插入。遍历完之后如果还有没处理的节点，将这些节点删除。

vue 优化比较的措施：优先处理特殊场景（头尾都没有变化/头尾互换），就地复用（vue 尽可能的复用 dom，尽可能不发生 dom 的移动，并且在特定索引下显示已被渲染过的元素）

在 vue 中这个移动是实时的，而 react 中则是维护一个队列，比较完之后再进行 dom 的操作，vue 中有一些措施针对这个实时的 dom 移动，所以性能方面没有影响。

4 父子组件通信

父传子：props

子传父：子组件用$emit()发送事件到父组件，父组件在子组件的标签上讲这个事件绑定到自己的方法。（子组件发布信息，父组件订阅信息）

5 非父子组件通信

中央事件总线（bus）

第一步：定义一个空的 vue 实例作为中央事件中心：var eventHub = new Vue()

第二步：在发送通信的组件里面用 emit 分发

第三步：在另外一个组件里面用 on 监听

第三步：组件销毁之前，要用 off 来销毁监听

6 父链

通过 this.$parent 可以直接访问该组件的父实例或者父组件。可以无限访问，直到根实例或者根组件。

不推荐使用，因为子组件应该尽可能的避免依赖父组件的数据，更不应该主动去修改他。这样会导致父子组件紧耦合。

7 子组件索引

通过 this.$children 可以访问子组件，但是因为子组件往往不止一个，很难区分。因此，用$refs 来代替。

父组件的子组件标签里，使用 refs 制定一个名称，就可以在子组件内用 this.$refs 来访问指定名称的子组件。用法：this.$refs.
名。子组件 <example ref='名'></example>

$refs 只在组件渲染完成后才填充，并且是非响应式的。因此避免在模版和计算属性中使用。

8 双向绑定原理

object.defineProperty()用来数据劫持，set 方法用来对对属性进行修改，get 方法用来对属性进行 return 抛出。

view-->data：数据监听。

data-->view：重点在与如何知道数据变化了。（数据劫持，设置 set 函数，一旦数据发生改变，就会触发这个函数）

监听器 observer：劫持监听所有属性，一旦发生变化就通知订阅者

订阅器 watcher：收到属性变化通知执行相关更新视图的操作

解析器 Compile：解析 html 上的 vue 标签，例如{{}}等

9 单向数据流

vue 中，子组件中一般不能直接修改父组件传过来的值，需要传回父组件，让父组件来改。

10.vue，react 和 angular 的区别

angular：运行效率较低，适合大型项目

react：更适合大型项目和更好的测试性，web 端和移动端原生 app

vue：更简单，适合小型的项目

11.filter

用法：属性|过滤器方法名

绑定方法：1.在 vue 实例中使用 filters:{名字:fnc}

2.全局定义过滤器，在 Vue 的构造函数上定义，然后再 new Vue()。Vue.filter('名字',fnc)

12.watch 和 computed

watch

7.2 vuex

1.什么是 vuex

vuex 是专门为 vue.js 应用程序开发的状态管理机制，一般用在大型的 spa 中。
处理的是组件之间的通信。

2.vuex 的模块

使用的时候需要在 main.js 中实例初始化的时候绑定 store，之后可以通过$store 调用
state：相当于 data，其中的数据就是需要通信的数据。$store.state
mutation：用来改变 state（改变 state）。$store.commit('mutation 中的方法')
action：可以用来改变 state（处理逻辑，然后调用 mutation）。在组件中$store.dispatch()，在 action 的方法中，使用 commit()。一般用来处理异步
getter：相当于计算属性。比如用来过滤 state 中数组中的元素小于 10
moduls：用来做多个独立的 state+mutation+...

7.3 vue-router

1.使用

更新视图而不重新请求页面

```
1.定义组件
2.定义路由
const routes = {
  {path: '/foo',component: Foo},
  {path: '/bar',component: Bar},
}
3.创建 router 实例
const router = new VueRouter({
  routes
})
4.创建和挂载实例
const app=new Vue({
  router
})
//代码中使用
this.$router 可以访问路由器
this.$route 可以访问当前路由
```

2.history 对象

是 window 对象的一部分
length：返回浏览器列表中的 url 数量
back()：加载前一个 url（go(-1)）
forwrad(): 加载下一个 url
go()：加载某个具体页面
pushState(state,title,url)：state 状态({page:1})，url 跳转的 url(必须同源),title 可以携带参数，但是有的浏览器不支持，所以最好不用 title，用 null 代替

3.原理

默认使用 hash，如果浏览器支持 history 则使用 history

7.3 ssr

1.什么是服务端渲染 ssr，为什么要用

由于 spa 单页应用中，是在客户端渲染的，服务端返回的是 json、xml 或者是 js，1.对于爬虫来说非常不利，需要客户端渲染（从 js 里生成 html），2.加载时间比较长。

渲染可以理解成一种修改，对 html 的修改（原本的初始 html，通过 js 修改，或者后端修改）。或者可以理解为将数据和模板组成 html。

ssr 服务端渲染，在 webpack 打包成 bundle.js 的时候，生成两份，一份给客户端，一份给服务端，他们渲染同一份 bundle.js，所以又叫同构。服务端渲染的时间比客户端要快，返回的也是 html，浏览器直接解析 html 即可。

ssr 的时候，必须要提供模板，把要填充的地方给后端，后端渲染返回 html。

2.前端渲染和后端渲染的区别，各自的好处

前端渲染：1.局部刷新无需每次请求都进行完整页面请求；2.富交互：可以使用 js 实现很多效果；3：节约服务器成本；4.前后端分离：后端只提供 json，不修改任何前端的代码。

后端渲染：1.加载时间优化，bundle.js 太大，不许下载。2.seo，可以呈现 html；3.减少客户端性能消耗。

3.ssr 可能遇到的问题

1.服务端没有 BOM（cookie、location 当前 url），服务端需要 mock window。而且因为 window 不一样，每次都要更新 mock。一些信息获取不到，例如之前项目中，使用了 env.isMT，但是前端环境中和后端环境中的是不一样的，所以造成前后端的渲染内容不一致的问题。

2.内存溢出。前端代码在浏览器中只要刷新就能够内存重置，ssr 不行。

3.性能。把百万台浏览器上的工作拿到服务器上来做，会对服务端的性能有很大的要求。

4.比较 nuxt 和 lego（koa 上层）

lego 框架是基于 koa 上层的框架，封装内部的一些方法（检测 cat 等）。主要关注的是 server 层。前端的渲染交给 vue 或者其他的来做。

nuxt 是 vue 封装的关注 UI 的框架，就是将前端的一些渲染封装起来，server 还需要 node 来操作。

他们是不同层次上的东西，lego 做后端，nuxt 做前端。nuxt 和 node 的前端都是用 vue-server-render 来讲 vue 实例转化为 html 的。

7.4 nuxt

1.什么是 nuxt

nuxt 是一个基于 vue.js 的通用应用框架，其实就是假设在 ssr 环境下前端应该用到的一些操作和工具封装起来。使用原生的 vue 也可以实现。关注的是 UI 的渲染。

nuxt = vue2 + Vue-Router + Vuex + Vue-Meta 等等

2.nuxt 静态化

3.nuxt 各个结构及其作用

1.assets：存放未编译的静态资源例如 less、js 等

2.components：vue 组件，不会 asyncData

3.layouts：组织应用的布局文件

4.pages：组织应用的路由及视图。自动读取该目录下的 vue 文件，生成路由配置

5.plugins：组织在根 vue 应用实例化之前需要运行的 js 插件

6.store：vuex，创建 index.JS 即可激活

nuxt.config.js：nuxt 配置

package.json：暴露脚本接口和依赖关系

7.5 规范

1.require 和 import 的区别

require 是同步的，require 运行完毕才会运行下面的代码。如果在客户端网络差，整个应用就会停在那里。因为内联的 js 会阻塞所有内容的显示和

2.客户端和服务端的差异

加载模块时，客户端主要受带宽的限制，服务端则因为在本地速度很快

3.amd、cmd、common.js

common.js

4.defer、async

八.其他框架

8.1 bootStrap,MINT-UI,ELEMENT

1.优点

（1）提供了非常丰富的组件和插件，例如导航、按钮组等，并接受定制
（2）拥有现成的 UI 组件，可以快速搭建网页页面
（3）流式栅格系统，响应移动设备优先

2.缺点

（1）定制会产生大量代码冗余，被覆盖的 css 代码没有价值但会占空间
（2）会有兼容问题，对低版本的浏览器兼容不好
（3）Bootstrap 基于 Less 构建，不支持 sass
（4）对于庞大的项目会产生奇怪的问题，需要花费大量时间来修改

3.bootstrap 响应式布局原理

采取 12 列的栅格布局，每段宽度一定，使用百分比和媒体查询实现。
设置 container 的 margi:auto 和 padding，根据媒体查询设置 container 的 width 属性。
再设置 col-xs-12 和其他的中 width：100%/或者计算成其他 11/12

8.2 微信小程序

1.生命周期

onload->onshow->onready

原生: onready（文档结构加载完成，不包括图片等非文字媒体文件）->onload（包括图片在内的所有元素都加载完成）。

2.获取设备信息

wx.getSystemInfo(callback)

3.获取用户信息

wx.getUserInfo(callback)

九.前端综合性问题

9.1 实习程序设计题

1.点击加载更多分页

组件接受三个参数，分别是 allDone（是否结束了）、currentPageNo（现在的页号），itemCount（list 总长度）

在组件内部定义变量 targetPageNo，起始设为 1。父组件中传递的 currentPageNo 初始为 0。methods 会 emit 一个方法，父组件接受这个方法，并且在这个方法中获取数据，并且修改 pageNo、allDone 等。

子组件中，初始化的时候调用 init 方法，就是将 targetPageNo 设置为 currentPageNo，然后 emit 发送给父组件（这里 targetPageNo++）。不是初始化的时候，在按钮上定义 click 方法，判断是否在 loading 中，如果是的话则点击无效，不是的话 emit 发送方法。在 watch 和 computed 中，计算 isLoading（当 current 不等于 current）。

9.2 项目实际应用

1.开发调试(web 端和移动端)

PC 网页：

十.计算机基础

10.1 操作系统

1.线程和进程的区别

进程和线程都是 cpu 工作时间段的描述。

进程是系统资源分配的最小单位。（CPU+内存+文件+网络等等资源）

线程是 cpu 调度的最小单位。（利用 CPU 中央处理器去运行代码，只管理 CPU 的那部分资源）。（各种调度算法）

2.什么是进程的同步和互斥

操作系统一般是多任务操作系统，即同时有大量和调度实体在运行。可能会导致两个问题：同时运行多个任务需要访问同一资源；同时运行的多个任务之间有依赖关系，一个任务的运行依赖于另外一个任务。互斥和同步就是用来解决这两个问题的。

互斥：两个任务不可以同时运行，他们会相互排斥，必须等待一个进程执行完毕，才能运行另外一个。互斥是一种特殊的同步。（唯一性和排他性，任务无序）

同步：也是不能同时运行，但他必须按照某种次序来运行响应的进程。同步是一种更为复杂的互斥。（顺序）

3.什么是临界资源，临界区访问原则。

只允许一个进程访问的共享资源成为临界资源，许多物理设备，例如打印机等都是，很多变量也是。

（1）空闲让进（没有进程在临界区，那么想进入临界区的资源就可以进入）

（2）忙则等待（当有资源在临界区时，其他欲进入的进程必须等待）

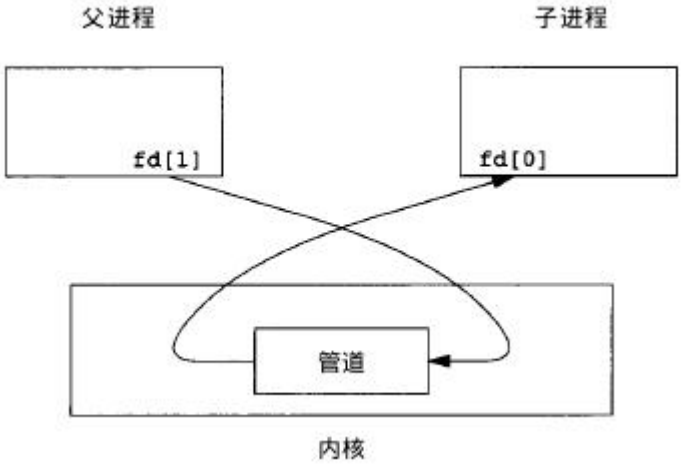
（3）有限等待（进入临界区的资源应该在有限时间内完成并退出临界区，不能让其他进程无限等待）

（4）让权等待（如果资源不能进入自己的临界区，则停止运行，让出处理器）

3.进程通信方式

（1）管道

半双工的，数据只在一个方向上流动，具有固定的读端和写端，相当于一种特殊的文件形式。



sd[0]为读打开，fd[1]为写而打开。

特点：速度慢，容量有限，只有父子进程能通讯

（2）FIFO

命名管道，是一种文件类型。

特点：任何进程间都能通信，但速度慢

（3）消息队列

在消息传输的传输过程中保存消息的容器。

（4）信号量

信号量是用来同步的，若是用来进程间通信需要结合共享内存。

特点：不能传递复杂信息，只能用来同步和传递简单信息

（5）共享内存

两个或多个进程共享一个给定的存储区（这是最快的 IPC，因为进程是直接对内存进行读取）

（因为多个进程可以同步操作，所以需要进行同步，一般用信号量来同步对内存的访问）
（6）sockt 网络

4.死锁与死锁产生的条件

两个或两个以上的进程在执行过程中，由于资源竞争或者彼此通信造成的一种阻塞状态，如果没有外力，他们将无法推进下去。

条件：（1）互斥，资源是独占的。
（2）不可剥夺，资源所获得的资源只能由该资源释放
（3）请求和保持
（4）循环等待

5.进程有哪几种状态

执行态（IO 请求--->阻塞）（时间片完--->就绪）
就绪态（调度--->执行）
阻塞态（IO 完成--->就绪）

6.线程同步的几种方式

（1）临界区，当多个线程访问一个独占性资源时，可以使用临界区对象，拥有临界区的进程可以访问被保护的资源。是目态的。
（2）互斥量
互斥量对象中有一个线程 id 属性，如果值为 0，则代表这个互斥量没有线程在占用，如果值为 1，则代表有线程在占用。是管态的。
（3）信号量
最大资源计数（创建的时候就设定好的）和可用资源计数。
（4）自动重置事件对象
（5）人工重置事件对象

6.分页和分段的区别

7.操作系统进程调度的方式有哪些

（1）先来先服务。
（2）短进程优先，估计各个进程的运行时间，选择最短的。
（3）最高优先权调度算法
（4）时间片轮转法，将多个进程按先来先服务排列成一个队列，每次调度，将 cpu 分配给队首进程，命令其执行一个时间片。
（5）多级反馈队列算法，设置多个就绪队列，后面的队列要比前一个的时间片长一倍，仅当前一个队列为空时，才为后一个队列分配。
8.死锁的处理基本策略和常用方法

2.cache 和 buffer

cache 为高速缓冲存储器，位于 cpu 和主存储之间，规模很小但速度很高。用来解决 CPU 和内存之间存取速度不一致的问题。用来读
Buffer:用来写

3.cpu 的目态和管态

管态称为特权状态，操作系统在管态下运行。
目态称为用户态，用户程序只能在目态中执行。

4.中断和异常

中断：在程序执行过程中，为了响应从硬件发来的信号而随机产生的事件，比如说外设的请求。
异常：处理器检测到执行一条指令时的错误，比如缺页故障、内部机器错误等。

5.GPU 和 CPU 的区别

CPU 由专门设计的几个串行的核心组成，用来计算个数比较少的复杂的问题。
GPU 是有多个比较简单的计算单元，用来计算个数很多的简单问题。

10.2 数据结构

1.B 树的特性，B 树和 B+树的区别

2.二叉树查找

前序：根节点-->左-->右
中序：左-->根节点-->右
后序：左-->右-->根节点

3.堆和栈的区别（java）

堆：先进先出
栈：先进后出
堆内存：对象和数组，new 出来的东西
栈内存：局部变量，更新速度很快

4.utf-8 和 unicode 的区别

unicode 是字符集，为每个字符分配一个　唯一的 id（码位）（例如 30693，计作 U+77E5，十六进制）

utf-8 是编码规则，将一个码位编译为字节序列，也就是一个二进制，最后存在计算机

unicode 是为了解决不同编码造成的问题，所有的语言的每个字都有唯一的编号，而 utf-8 采用可变长的方式，将 unicode 转化为二进制，用一个字节保存在计算机里。

10.3 MySql

10.4 设计模式

1.面向对象设计的基本原则

(1) 单一职责原则：一个类处理一个功能。

(2) 开闭原则：对扩展开放，对修改关闭。即在设计一个模块的时候，应当使这个模块可以在不被修改的情况下扩展。

(3) 里氏替换原则：任何基类出现的地方，子类是完全可以来替换它的

(4) 依赖倒置原则：针对抽象编程，不要针对具体编程

(5) 接口分离原则：

(6)

(7)

2.观察者模式

3.工厂模式

4.策略模式

5.原型模式

10.5 linux

1.linux 服务器性能下降，怎么排查解决

2.相关命令

dmesg 网卡或硬盘有问题时，用来查看
netstat 查看网络信息
ps 查看进程信息
free 查看 cpu 信息

10.6 算法题

1.各种排序

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性	复杂性
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	简单
希尔排序	$O(nlog_2n)$	$O(n^2)$	$O(n)$	$O(1)$	不稳定	较复杂
直接选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	简单
堆排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(1)$	不稳定	较复杂
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	简单
快速排序	$O(nlog_2n)$	$O(n^2)$	$O(nlog_2n)$	$O(nlog_2n)$	不稳定	较复杂
归并排序	$O(nlog_2n)$	$O(nlog_2n)$	$O(nlog_2n)$	$O(n)$	稳定	较复杂
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(n+r)$	稳定	较复杂

2.不稳定性指的是什么

不稳定性指的是当有相同值存在时，最后排序的结果，相同元素的下标紊乱

10.7 git

1.上传代码

step1: 暂存代码.git add .
step2: git commit -m “提交信息”（提交到本地仓库）
step3: push 到远程分支 git push -u origin master

2.拉取非 master

git clone -b 远程分支名 远程地址

3.解决冲突

多人同时修改同一个项目，本地的代码相对于远程 master 过旧：拉取远程 master 与本地合并 git
git pull 远程分支：本地分支

4.其他 git 命令

git checkout dd 切换分支到 dd
git checkout -b dd 切换到新分支 dd（有直接切换，没有则新建）
git branch dd 创建分支 dd

5.git pull 发生了什么

假设本地是在 master 分支上
git fetch origin maste:tmp 在本地新建一个 tmp 分支，将远程分支的代码下载到 tmp 分支上
git diff tmp 比较本地的 master 分支和远程的 master 分支的区别
git merge tmp 合并 tmp 分支到本地的 master 分支上
git branch -d tmp 不想保留 tmp 分支，进行删除

6.git 工作流

开发新功能，从远程 master 新建分支，在这个分支上开发。本地开发完成之后，推送到远程的分支，然后远程分支与 master 分支合并发布。